

Pseudocódigo

Edelmira Rodríguez Alcántar

Pseudocódigo con PSeInt

Edelmira Rodríguez Alcántar



"El saber de mis hijos
hará mi grandeza"

Índice general

	Página
1. Introducción	1
2. Elementos básicos	3
2.1. Identificadores	4
2.2. Tipos de datos simples	6
2.3. Declaración de variables	7
2.4. Escritura de datos	8
2.5. Lectura de variables	10
2.6. Primer algoritmo secuencial	12
2.7. Operadores aritméticos	17
2.8. Operador aritmético MOD	25
2.9. Operadores relacionales	26
2.10. Operadores lógicos	28
2.11. Leyes de DeMorgan	29
3. Selección	33

3.1. Selección simple	33
3.2. Selección doble	40
3.3. Selección múltiple	59
4. Repetición	73
4.1. Ciclo Mientras Hacer	73
4.1.1. Variantes del uso del ciclo	92
4.2. Ciclo Para-Hacer	98
4.2.1. Figuras geométricas	105
4.3. Repetir-Hasta que	110
Referencias	119
Lista de algoritmos	121
Lista de cuadros	125
Lista de figuras	127

Capítulo 1 | Introducción

Cuando se desea solucionar un problema particular es esencial tener una completa comprensión del mismo y planear una estrategia para atacarlo.

La solución de cualquier problema de cómputo involucra la ejecución de una serie de acciones en un orden determinado. Un algoritmo es un procedimiento para resolver un problema en términos de las acciones a realizar y el orden en que deben ejecutarse estas acciones.

Cuando queremos resolver un problema por medio de la computadora, debemos plantearlo primero en nuestras palabras para asegurarnos de comprenderlo antes de pasarlo a un lenguaje de programación específico donde se nos pedirá que, además de especificar las acciones que resolverán el problema y el orden en que se ejecutarán, cuidemos las reglas de ortografía y sintaxis del lenguaje particular (por ejemplo, C/C++, Python, etc.). En este trabajo se presenta una técnica para describir los pasos que llevarán a la resolución de un problema: el pseudocódigo.

El pseudocódigo tiene la ventaja de ser un lenguaje intermedio entre el lenguaje natural y un lenguaje de computadoras, es decir, por un lado es lo suficientemente parecido al lenguaje natural (español en este caso) por lo que no se tienen dificultades de comprensión; y por otro, requiere de la utilización de ciertas reglas que nos ayudan a expresar las ideas en una forma lógica sin cuidar los detalles sintácticos de un lenguaje de programación.

Para facilitar el aprendizaje, se utiliza el software PSeInt, un IDE donde se permite la edición del algoritmo, así como su compilación y ejecución. Es necesario seguir ciertas reglas, que de momento pueden parecer tedio-

sas sin embargo, nos van a permitir ordenar mejor nuestras ideas al diseñar un algoritmo en pseudocódigo. Además, se aprovecha que PSeInt tiene una herramienta para trabajar con diagramas de flujo.

En el Capítulo 2 se presentan los elementos básicos de la programación en pseudocódigo: los tipos de datos que utilizaremos, la forma de declararlos, las operaciones aritméticas elementales y las operaciones de lectura y escritura de datos, así como algunos operadores adicionales.

En el capítulo 3, se introduce la programación estructurada estudiando las estructuras de selección (simple, doble y múltiple) y en el capítulo 4 se presentan las estructuras de repetición (mientras-hacer, para-hacer y hacer-mientras).

Esperamos que este trabajo sirva de apoyo para aquellos que están aprendiendo a programar en la clase de Diseño de Algoritmos de la Licenciatura en Ciencias de la Computación.

Capítulo 2 | Elementos básicos

Cuando utilizamos la computadora para resolver un problema, generalmente debemos proporcionarle los valores a partir de los cuales queremos que nos arroje una solución. Estos valores son llamados la entrada del programa. Los valores que se generan como resultado de operar con los valores de entrada son llamados la salida del programa. Hay un conjunto de operaciones que se consideran básicas para manejar los valores de entrada y de salida de forma que resuelvan nuestros problemas. Una vez conocidos estos elementos, estaremos en condiciones de escribir nuestro primer algoritmo en pseudocódigo. Estas operaciones básicas son: Declaración de variables, definición de constantes, lectura y escritura de datos, así como las operaciones aritméticas, relacionales y lógicas que se pueden utilizar.

Hablaremos primero de la estructura de los datos que se utilizarán, es decir, la forma de representar la información real del problema para que pueda ser almacenada y manipulada por la computadora.

Un dato se presenta en forma de variable en un algoritmo. Una variable tiene tres atributos:

1. **Nombre o identificador.** Permite identificar un dato para diferenciarlo de otros y manipularlo.
2. **Contenido.** Es el valor que contiene el dato.
3. **Tipo.** Tipo de dato asociado.

En el resto del capítulo se describe cada uno de los puntos mencionados.

2.1. Identificadores

Los nombres usados para hacer referencia a las variables, constantes, funciones, etiquetas y otros objetos definidos en un programa se conocen como *identificadores*. Para crear un identificador, se deben tomar en cuenta las siguientes reglas ¹:

1. Únicamente pueden utilizarse letras mayúsculas y minúsculas (a...z, A...Z), dígitos (0...9) y/o el carácter de subrayado (_).
2. Puede iniciarse con letra o el carácter de subrayado (es decir, no puede iniciarse un identificador con un dígito).
3. No deben ser palabras reservadas del PSeInt.
4. No se distingue entre mayúsculas y minúsculas.

El listado de palabras reservadas más comunes de PseInt se muestra en el Cuadro 2.1.

Ejemplos de identificadores válidos:

- suma_enteros
- x
- resp_usuario
- _c1
- c2
- c_1

Ejemplos de identificadores no válidos (a la derecha se especifica por qué no son válidos):

¹En pseudocódigo no existen reglas específicas para la nomenclatura de identificadores, sin embargo, convendremos en utilizar las presentadas en esta sección por compatibilidad con el lenguaje C/C++ y Python

Cuadro 2.1: Palabras reservadas del PseInt

Algoritmo	FinAlgoritmo	Proceso	FinProceso
Definir	Como	Entero	Enteros
Caracter	Caracteres	Cadena	Cadenas
Leer	Si	Entonces	SiNo
FinSi	Segun	Hacer	De Otro Modo
FinSegun	Mientras	FinMientras	Repetir
Hasta Que	Para	Con Paso	FinPara
Funcion	FinFuncion	Fin	Verdadero
Falso	Y	O	No
Mod	Div	Limpiar Pantalla	Esperar Tecla
Dimension	Mostrar	Imprimir	es
Son	Real	Reales	Texto
Logico	Logicos	Numero	Numerico
Logica	Logicas	Entera	Enteras
Escribir	Sin	Saltar	Limpiar
Pantalla	opcion	valor	

- suma enteros $\leftarrow$ *problema: el espacio*
- año $\leftarrow$ *problema: la ñ*
- calificación $\leftarrow$ *problema: la letra acentuada*
- 2C $\leftarrow$ *problema: iniciar con un dígito*

Favor de contestar el formulario que se encuentra en la siguiente dirección [Formulario1](#). $\leftarrow$

La información de entrada, de salida y la generada durante el proceso de solución debe almacenarse y ser accesible en el momento que sea requerida a través de las variables y/o constantes que tendrán asociado un tipo de dato. Los diferentes tipos de datos aceptados se mencionan en la siguiente sección.

2.2. Tipos de datos simples

El PSeInt soporta tres tipos de datos simples: numérico, lógico y cadena.

Numérico Se utiliza almacenar valores numéricos que pueden ser enteros o reales. Los números reales utilizan el punto para separar los decimales. **Número**, **numérico** y **real**, son sinónimos para definir este tipo de dato. Hay un tipo de dato numérico especial, **entero**, que permite almacenar solo valores enteros; si un valor no entero le es asignado, se trunca para convertirlo a entero. Si se usa una sola instrucción para definir más de una variable entera, se permite en uso del plural.

Lógico Una variable declarada de tipo LOGICO, podrá contener solo los valores VERDADERO y FALSO, aunque el usuario también puede usar los valores F y V ó 0 y 1, durante la captura de datos, en cuyo caso, se convertirán en VERDADERO o FALSO, según corresponda.

Cadena Las variables de este tipo pueden contener un carácter o una cadena de caracteres encerrados entre comillas (las comillas pueden ser simples o dobles). **Caracter**, **texto** y **cadena** son sinónimos para definir este tipo de dato.

En resumen, los tipos de datos posibles son **número**, **numérico**, **real**, **entero**, **caracter**, **texto** y **cadena**. Cabe señalar que si se intenta asignar un valor de tipo incorrecto a una variable ya definida, se producirá un error al tiempo de ejecución.

PSeInt no es sensitivo, eso significa que pueden usarse mayúsculas o minúsculas en la definición y uso del nombre y tipo de variables.

Ejemplos de valores posibles en cada tipo de dato:

- Entero *1, 4, -6, 342, -32021, 0*
- Real *-238.423, 2.0, 3.1542, 0.00001*

- Caracter "{", "*", "9", " *Universidad de Sonora*", " *Mi teléfono es +52 (662) 259-21-55*", 'Hola', 'S'

Para poder utilizar una variable en un algoritmo, es necesario declararla.

2.3. Declaración de variables

Cuando declaramos una variable, le estamos anunciando a la computadora qué tipo de datos vamos a almacenar en dicha variable. El formato para la declaración de variables es:

```
Definir <var1> Como [tipo singular]
Definir <var1, var2, ...> Como [tipo plural]
```

donde, <var1> se refiere al identificador de la variable y con [tipo] se está indicando que debe seleccionarse uno de los tipos de datos válidos.

Por ejemplo, supongamos que requerimos la edad de una persona en años y que para ello, usamos el identificador edad. La declaración del identificador se hará como sigue:

```
Definir edad Como Entero
```

Si se trata de más de una variable de tipo Entero, puede usarse

```
Definir num1, num2 Como Enteros
```

Si requerimos conocer el peso de un objeto o de una persona, es posible usar el identificador peso y declararlo de la siguiente manera:

```
Definir peso Como Real
```

Si hay más de una variable real, puede indicarse como

```
Definir num1, num2 Como Reales
```

Si preguntamos al usuario si quiere terminar el programa y esperamos una respuesta que puede ser 's' o 'n' (que represente *si* o *no*, respectivamente), declaramos un identificador de tipo caracter, como por ejemplo:

```
Definir resp Como Caracter
```

Si requerimos conocer el nombre del usuario, podemos declarar un dato de tipo cadena de caracteres de la forma siguiente:

```
| Definir resp Como Cadena
```

aunque también es aceptada la siguiente declaración

```
| Definir resp Como Caracteres
```

La sintaxis flexible permite definir tipos de datos de forma alternativa mediante las palabras clave **Es** y **Son**, como sigue

```
| <var1> Es [Real/Entero/Logico/Caracter]
| <var1, var2, ...> Son [Reales/Enteros/Logicos/Caracteres]
```

Se sugiere usar identificadores que estén relacionados con lo que representarán en el algoritmo, por ejemplo, si se quieren declarar una variable para almacenar una calificación es posible usar uno de los identificadores siguientes: `calif` o bien, `calificacion`; esto hará que el algoritmo sea más legible.

La interacción usuario-máquina se desarrolla mediante el envío de mensajes por parte de la máquina y de la introducción de información (datos) por parte del usuario. Los mensajes de la computadora tienen el propósito de guiar al usuario en el uso del programa, para lo cual pueden utilizarse mensajes descriptivos del funcionamiento del mismo, de solicitud de información y/o de presentación de resultados. El usuario proporciona información siguiendo las instrucciones de los mensajes proporcionados.

Esta interacción requiere de dos operaciones: la escritura de datos y la lectura de variables, las cuales se verán en las secciones 2.4 y 2.5, respectivamente.

2.4. Escritura de datos

Esta operación permite a la máquina presentar datos al usuario. El formato que usaremos es el siguiente:

```
| Escribir dato
```

donde *Escribir* especifica la operación a realizar y 'dato' contiene la información que se desea presentar al usuario.

Por ejemplo, si se desea presentar el mensaje "Hola, UniSon", se utiliza la siguiente instrucción

```
| Escribir "Hola, UniSon"
```

en este caso la información presentada es una cadena constante sin nombre.

El símbolo usado en el PSeInt para construir el diagrama de flujo correspondiente a la lectura se muestra en la Figura 2.1.

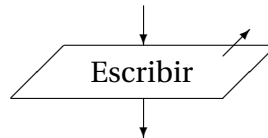


Figura 2.1: Diagrama para la lectura de datos.

Es posible enviar varios datos con la misma instrucción para lo cual es necesario separarlos con comas. Por ejemplo, la instrucción

```
| Escribir "Hola, UniSon", "Soy alumno de primer ingreso"
```

está enviando dos cadenas "Hola, UniSon" y "Soy alumno de primer ingreso". Esto es equivalente a utilizar dos instrucciones de escritura; es decir,

```
| Escribir "Hola, UniSon"  
| Escribir "Soy alumno de primer ingreso"
```

tiene el mismo efecto que la sentencia compuesta anterior.

Puede combinarse la impresión de datos variables y constantes. Por ejemplo,

```
| Escribir "Tu edad es", edad
```

está imprimiendo la cadena constante "Tu edad es" y el contenido de la variable edad.

Obsérvese que cada una de las llamadas a **Escribir** contiene una instrucción de salto de línea al finalizar la impresión. Podemos alterar esto mediante el comando **Sin Saltar**, de la siguiente manera

```
Escribir "Hola, UniSon" Sin Saltar  
Escribir "Soy alumno de primer ingreso"
```

Con el comando Sin Saltar estamos indicando que el cursor debe quedar en el mismo renglón al finalizar la impresión, con lo cual, al llamar a la siguiente instrucción de escritura, se iniciará en el mismo renglón. Este funcionamiento es adecuado cuando estamos solicitando información, por ejemplo, en el siguiente segmento de código se utiliza una sola línea para la solicitud y lectura de la edad de una persona.

```
Definir edad como entero  
Escribir "Cuántos años tienes? " Sin Saltar  
Leer edad
```

2.5. Lectura de variables

Es la operación mediante la cual se captura, o se recibe, la información proporcionada por el usuario. El formato es el siguiente

```
Leer variable
```

donde Leer indica la operación a realizar y *variable* es el nombre de la variable donde se almacenará la información leída. Por ejemplo, si el usuario proporcionará la edad de una persona (y declaramos la variable edad para recibirla) esta información puede ser capturada por el algoritmo mediante la instrucción

```
Leer edad
```

El símbolo utilizado en el diagrama de flujo para la operación de lectura es el que se muestra en la Figura 2.2.

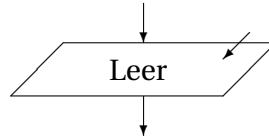


Figura 2.2: Diagrama para la lectura de datos.

Generalmente, la lectura de variables va precedida de algún mensaje indicando al usuario la información que se está esperando, es decir, la lectura de variables va precedida de una operación de escritura. Por ejemplo:

```
Escribir "Qué edad tienes?"  
Leer edad
```

Cuando se requiera leer varios valores puede usarse una instrucción de lectura para cada uno de ellos, o bien, utilizar sólo una instrucción de lectura separando con comas las variables donde se almacenarán los datos. Por ejemplo, el bloque de instrucciones siguiente

```
Escribir "Teclea dos calificaciones"  
Leer c1  
Leer c2
```

y el siguiente

```
Escribir "Teclea dos calificaciones"  
Leer c1,c2
```

son equivalentes.

También es posible solicitar la información por separado, por ejemplo,

```
Escribir "Teclear la calificacion 1"  
Leer c1  
  
Escribir "Teclear la calificacion 2"  
Leer c2
```

En este momento, tenemos los elementos necesarios para presentar nuestro primer algoritmo secuencial.

2.6. Primer algoritmo secuencial

La estructura general de los algoritmos que aquí se desarrollan consta de 2 partes: en la primera se definen los datos de tipo variable y en la segunda, se enlistan las instrucciones que se realizarán. El formato mencionado se ilustra a continuación:

```
1 // Nombre:
2 // Autor:
3 // Fecha:
4 // Descripción:
5 Algoritmo NombreAlgoritmo
6   [Definición de variables]
7
8   [Conjunto de instrucciones]
9 FinAlgoritmo
```

Es posible incluir comentarios en el código, es decir, notas explicativas pensando en un lector posterior (que puede ser el mismo programador en el futuro). El uso de comentarios es una estrategia para hacer más legible el código sin afectar su funcionamiento, pues se trata de texto que no se ejecuta. Cuando se inserta el símbolo //, lo que queda a la derecha se considera un comentario, hasta el final de la línea.

A manera de formato, en todos los algoritmos se incluye un encabezado con el nombre del algoritmo, el nombre del autor, la fecha de elaboración y una breve descripción del funcionamiento. Todo lo demás estará delimitado por las instrucciones **Algoritmo** y **FinAlgoritmo** con las cuales se indica dónde comienza y dónde termina el algoritmo, respectivamente. Todas las instrucciones que conforman el algoritmo están desfasadas un espacio a la derecha con respecto a los delimitadores (inicio y fin) lo cual mejora la legibilidad del algoritmo. A este espacio se le llama **sangría**.

La parte de la definición de variables que conforman el algoritmo se encuentra entre corchetes lo cual significa que no necesariamente se utilizarán, es decir, es posible que no se requiera variable alguna y, en tal caso, se omite la sección. El Ejemplo 2.1 presenta un algoritmo sencillo que ilustra la utilización de este formato.

Ejemplo 2.1. Solicitar y leer 3 datos de diferente tipo e imprimirlos.

La información requerida puede ser el nombre del usuario (cadena), su edad (entero) y su peso (real).

Para ilustrar el hecho de que es posible realizar las mismas instrucciones de diferentes maneras se presentan tres versiones de este algoritmo.

En la primera versión, Algoritmo 2.1, se solicitan y se leen por separado cada uno de los datos y se imprimen utilizando solo una instrucción de escritura.

Algoritmo 2.1: Solicitud y captura de datos

```

1 //Nombre: Lectura_datos_a.txt
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 //Descripción: En este algoritmo se declaran tres variables de distinto tipo.
4 // Se solicita al usuario que capture en valor de cada una
5 // y se guarda cada valor en la variable adecuada.
6
7 Algoritmo Lectura_datos_a
8   Definir nombre Como Cadena
9   Definir edad Como Entero
10  Definir peso Como Real
11
12  Escribir "Cuál es tu nombre?" Sin Saltar
13  Leer nombre
14  Escribir "Cuántos años tienes?" Sin Saltar
15  Leer edad
16  Escribir "Cuánto pesas?" Sin Saltar
17  Leer peso
18
19  Escribir "Hola ", nombre, ", tienes ", edad, " años y pesas ", peso, " kilos"
20 FinAlgoritmo

```

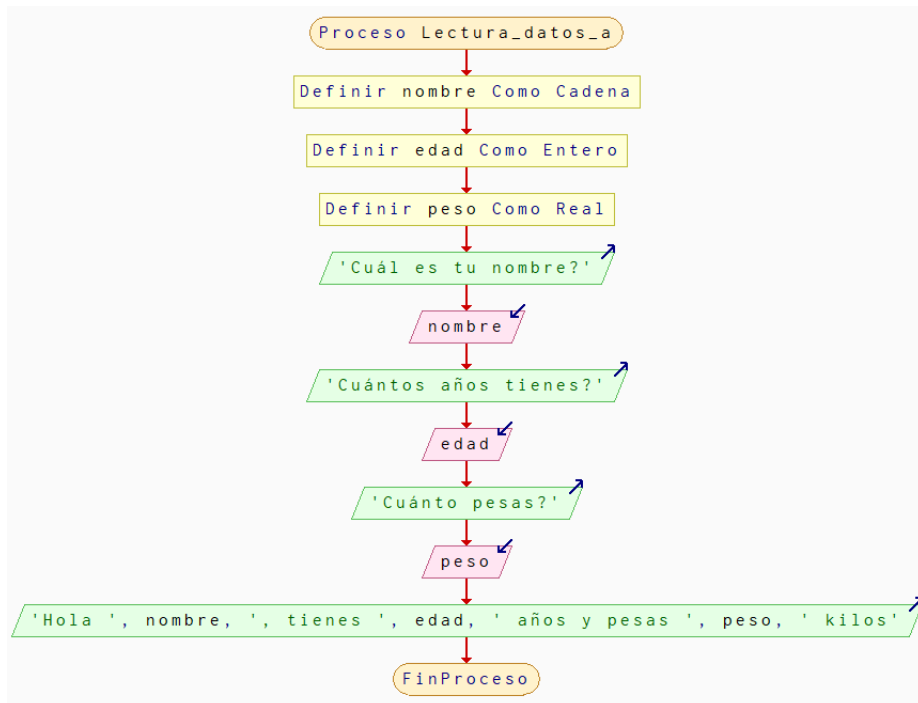


Figura 2.3: Diagrama de flujo para el Algoritmo 2.1.

En el Algoritmo 2.2, la edad y el peso del usuario se solicitan y leen con una sola instrucción de escritura y una de lectura, respectivamente. El resto de las instrucciones son idénticas a las presentadas en el Algoritmo 2.1.

Algoritmo 2.2: Solicitud y captura de datos

```
1 // Nombre: Lectura_datos_b.txt
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: En este algoritmo se declaran tres variables de distinto tipo.
4 //   Se solicita al usuario que capture en valor de cada una
5 //   y se guarda cada valor en la variable adecuada.
6
7 Algoritmo Lectura_datos_b
8   Definir nombre Como Cadena
9   Definir edad Como Entero
10  Definir peso Como Real
11
12  Escribir "Cuál es tu nombre?" Sin Saltar
13  Leer nombre
14  Escribir "Cuántos años tienes y cuánto pesas?"
15  Leer edad,peso
16
17  Escribir "Hola",nombre,"tienes", edad,"años y pesas", peso , "kilos"
18 FinAlgoritmo
```

En el Algoritmo 2.3, se solicitan y se leen los datos por separado y se imprimen utilizando varias instrucciones de escritura.

Algoritmo 2.3: Solicitud y captura de datos

```
1 // Nombre: Lectura_datos_c.txt
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: En este algoritmo se declaran tres variables de distinto tipo.
4 // Se solicita al usuario que capture en valor de cada una
5 // y se guarda cada valor en la variable adecuada.
6
7 Algoritmo Lectura_datos_c
8   Definir nombre Como Cadena
9   Definir edad Como Entero
10  Definir peso Como Real
11
12  Escribir "Cuál es tu nombre?" Sin Saltar
13  Leer nombre
14  Escribir "Cuántos años tienes?" Sin Saltar
15  Leer edad
16  Escribir "Cuánto pesas?" Sin Saltar
17  Leer peso
18
19  Escribir "Hola", nombre
20  Escribir "Tienes", edad, "años"
21  Escribir "Pesas", peso, "kilos"
22 FinAlgoritmo
```

Existen muchas maneras de escribir algoritmos que respondan adecuadamente a la misma necesidad; en realidad, es cuestión de estilo del programador.

Los algoritmos presentados hasta aquí son sumamente sencillos, útiles únicamente con fines ilustrativos del esquema que se utilizará en la escritura de algoritmos. Generalmente, se desarrolla un algoritmo para que resuelva algún problema a través de la realización de la manipulación de los valores de entrada. Por ello, es necesario conocer la forma en que se operará con dichos valores en un algoritmo. El resto del capítulo se dedica a la presentación de las operaciones aritméticas, relacionales y lógicas que usaremos para la manipulación de los datos de entrada en nuestros algoritmos.

2.7. Operadores aritméticos

Las operaciones aritméticas que manejaremos en pseudocódigo son: suma, resta, multiplicación, división y asignación. En el Cuadro 2.2 se enlistan las operaciones básicas y el operador que se utiliza para indicarlas.

Cuadro 2.2: Operadores aritméticos

Operación	Operador
Suma	+
Resta	−
Multiplicación	*
División (real)	/
Módulo	MOD
Asignación	:= <-

Todos los operadores se aplican en forma binaria (con dos operandos) y utilizan operandos constantes o variables a la izquierda y derecha del operador, según se requiera, excepto para el operador de asignación, el cual requiere de su lado izquierdo una variable.

El operador de *asignación* toma su nombre debido a que asigna el valor de la derecha a la variable de la izquierda, tal como se indica en el formato siguiente

```
| variable := valor asignado
```

Hay un símbolo en los diagramas de flujo para esta instrucción:

El valor asignado a la variable puede ser un valor constante o el resultado de aplicar diversas operaciones aritméticas.

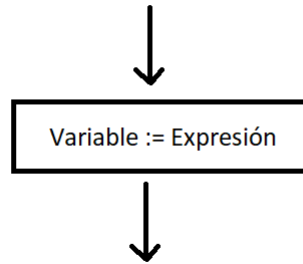


Figura 2.4: Símbolo de la asignación en los diagramas de flujo.

Por ejemplo,

<code>x := 2</code>	asigna el valor constante 2 a x
<code>x := y</code>	asigna el valor de la variable y a x
<code>x := 2+y</code>	asigna el resultado de la suma de 2 y el valor de la variable y a x
<code>x := aleatorio(1,5)</code>	asigna el valor regresado por la función <code>aleatorio()</code> a x
<code>letra := 'a'</code>	asigna el carácter 'a' a la variable letra
<code>cad := 'Irene'</code>	asigna la cadena 'Irene' a cad

Los operadores `+` y `-` también pueden ser aplicados en forma monaria, es decir, con un operando. Regularmente los operadores monarios se anteponen a su operando.

Por ejemplo,

<code>x1 := -2</code>	asigna el valor constante -2 a x1
<code>x2 := +3</code>	asigna el valor 3 a x2

Las operaciones deben escribirse en forma lineal, es decir, en una sola línea. Por ejemplo, si se tiene la fórmula

$$x = \frac{1}{a} + \frac{1}{b}$$

debe transformarse para que tome la forma

$$x = 1/a + 1/b$$

Hay ocasiones en que la transformación no es tan directa. Por ejemplo, la ecuación

$$x = \frac{a+b}{2}c$$

debe ser transformada para que ocupe una única línea, pero la división entre 2 y la multiplicación por c están afectando al factor a+b. Por lo tanto, estas operaciones deberán presentarse en la expresión entre paréntesis tal como se muestra a continuación:

$$x = (a + b) * c / 2$$

o bien

$$x = (a + b) / 2 * c$$

Para que seamos capaces de hacer la transformación correctamente, es necesario conocer la precedencia de los operadores (es decir, saber qué operador se ejecuta primero). En el Cuadro 2.3 se muestra la precedencia de los operadores mencionados, los que están más arriba en la tabla tienen mayor precedencia, los que tienen la misma precedencia se muestran en el mismo renglón separados por comas. Cuando en una expresión se tienen operadores de la misma precedencia se van ejecutando de izquierda a derecha, excepto con los operadores monarios y el operador de asignación. Si se desea modificar la precedencia, se utilizan los paréntesis.

Por ejemplo, la expresión

$$x = \frac{2 + 5a}{-\frac{a}{2} + ab}$$

se transforma en

Cuadro 2.3: Precedencia y asociatividad de los operadores aritméticos

Nivel de precedencia	Operadores	Asociatividad
1	()	
2	+ - (monarios)	Derecha a izquierda
3	* / MOD	Izquierda a derecha
4	+ - (binarios)	Izquierda a derecha
5	:=	Derecha a izquierda

$$x = (2 + 5 * a) / (-a / 2 + a * b)$$

donde los operadores de suma (+) están aplicándose como binarios y el operador de resta está siendo utilizado como monario.

Ejemplo 2.2. Calcular el área de un triángulo dadas la base y la altura.

Supóngase que se desea encontrar el área de un triángulo utilizando la fórmula

$$A = \frac{bh}{2}$$

para lo cual es necesario conocer los valores de la base y la altura, así que se solicitan al usuario y, una vez realizadas las operaciones indicadas, se obtiene el valor deseado. Se requieren 3 variables: una para almacenar el valor de la base, otra para el valor de la altura y una más para almacenar el resultado de la operación. Sean las variables mencionadas base, altura y area, respectivamente. Entonces, la expresión anterior se transforma en

```
| area := base * altura / 2
```

Obsérvese que no se requiere del uso de paréntesis, pues la precedencia de los operadores indica en qué orden se van a ejecutar las operaciones y coincide con el resultado esperado.

En el Algoritmo 2.4 se presentan las instrucciones que se requieren para resolver este problema.

Algoritmo 2.4: Área de un triángulo.

```
1 // Nombre: AreaTriangulo_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Se calcula el área de un triángulo, dadas la base y altura.
4
5 Proceso AreaTriangulo
6   // Se declaran las variables para almacenar las dimensiones del triángulo y su área
7   Definir base, altura, area Como Real
8
9   // Se indica al usuario el propósito del algoritmo
10  Escribir "Se calculará el área de un triángulo"
11
12  //Se capturan las dimensiones del triángulo
13  Escribir Sin Saltar "Base:"
14  Leer base
15
16  Escribir Sin Saltar "Altura:"
17  Leer altura
18
19  // Se calcula el área
20  area := base * altura / 2
21
22  // Se imprime el área
23  Escribir "El área del triángulo es ", area, " u^2"
24 FinProceso
```

Observamos que la variable area solo se utiliza como auxiliar para almacenar el resultado de la operación realizada. Podemos utilizar la expresión de la operación directamente en la instrucción de escritura, con lo cual nos ahorramos una variable. Así, la sentencia de impresión queda como sigue:

```
| escribir "El área del triángulo es", base*altura/2," u^2"
```

En el Algoritmo 2.5 se incluyen las modificaciones mencionadas.

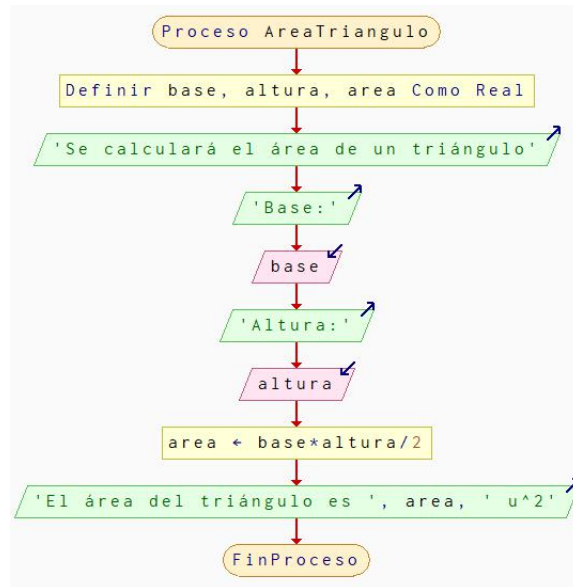


Figura 2.5: Diagrama de flujo para el Algoritmo 2.4.

Algoritmo 2.5: Área de un triángulo.

```

1 // Nombre: AreaTriangulo_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Fecha: 23 de agosto de 2019
4 // Descripción: Se calcula el área de un triángulo, dadas la base y altura.
5
6 Proceso AreaTriangulo
7 // Se declaran las variables para almacenar las dimensiones del triángulo y su área
8 Definir base, altura Como Real
9
10 // Se indica al usuario el propósito del algoritmo
11 Escribir "Se calculará el área de un triángulo"
12
13 //Se capturan las dimensiones del triángulo
14 Escribir Sin Saltar "Base:"
15 Leer base
16
17 Escribir Sin Saltar "Altura:"
18 Leer altura
19
20 // Se imprime el área
21 Escribir "El área del triángulo es ", base*altura/2, " u^2"
22 FinProceso
  
```

Ejemplo 2.3. Calcular el área de un círculo.

Ahora se desea obtener el área de un círculo mediante la ecuación

$$A = \pi * r^2$$

El usuario debe proporcionar el valor del radio (r) y el valor aproximado de π que se desee utilizar. Si se utilizan las variables `radio` y `aprox_pi`, respectivamente, la ecuación anterior se transforma en

```
| area := aprox_pi * radio * radio
```

A continuación, se muestran dos versiones de implementación. En el Algoritmo 2.6 se utilizan 3 variables, una para el radio, otra para el valor de π y una más, para el resultado.

Algoritmo 2.6: Área de un círculo.

```

1 //Nombre: AreaCirculo_a.txt
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 //Descripción:
4
5 Proceso
6   Definir radio, aprox_pi, area Como Real
7
8   Escribir "Valor del radio="
9   Leer radio
10  Escribir "Valor de pi="
11  Leer aprox_pi
12
13  area := aprox_pi*radio*radio
14
15  Escribir "El área del círculo con radio ",radio," es ",area
16 FinProceso

```

En el Algoritmo 2.7 se aprovecha que el valor de π está definido como una constante en PSeInt, de forma que solamente se solicita el valor del radio. El valor de π se obtiene usando la expresión PI.

Algoritmo 2.7: Área de un círculo.

```

1 // Nombre: AreaCirculo_b.txt
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Algoritmo
6   Definir radio, area Como Real
7
8   Escribir "Valor del radio="
9   Leer radio
10
11  area:= PI*radio*radio
12
13  Escribir "El área del círculo con radio ",radio," es ",area
14 FinAlgoritmo

```

En la siguiente sección presentaremos un operador aritmético adicional que será muy útiles en la solución de problemas en secciones posteriores.

2.8. Operador aritmético MOD

Este operador aritmético se utiliza para realizar divisiones con enteros cuyo resultado es también un entero. El operador MOD obtiene el residuo de la división entera. Por ejemplo:

```
| x := 7 MOD 3    // asigna 1 a x
```

Ejemplo 2.4. Usando división entera.

Algoritmo 2.8: Operaciones enteras.

```
1 // Nombre: OperacionesEntreEnteros.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Se ejemplifica el uso de operaciones entre números enteros.
4
5 Proceso OperacionesEntreEnteros
6   Definir x1,x2 Como Entero
7
8   Escribir "Proporciona un valor para x1"
9   Leer x1
10  Escribir "Proporciona un valor para x2"
11  Leer x2
12
13  Escribir "El cociente de la división entera entre ",x1," y ",x2," es ",trunc(x1/x2)
14  Escribir "El residuo de la división entera entre ",x1," y ",x2," es ",x1 mod x2
15 FinProceso
```

Ejemplo 2.5. Separar una cantidad en billetes de 100, monedas de 10 y de 1 peso.

Para empezar, se debe leer la cantidad que se desea descomponer, cantidad. Para separar en billetes de 100, hay que dividir la cantidad entre 100 con división entera, $\text{billetes100} := \text{trunc}(\text{cantidad}/100)$. Una vez obtenido el número de billetes de 100, hay que calcular el resto del dinero a feriar, con la instrucción $\text{resto} = \text{cantidad} \bmod 100$. Con el resto se calcula el número de monedas de 10 que se requieren usando división entera, $\text{monedas10} := \text{trunc}(\text{resto}/10)$. Finalmente, al calcular $\text{monedas1} = \text{resto} \bmod 10$, se obtiene el número de monedas de 1 peso requeridas.

Algoritmo 2.9: Operaciones enteras.

```

1 // Nombre: SeparaDinero.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Se separa una cantidad en billetes de 100, monedas de 10 y de 1 peso
4
5 Proceso SeparaDinero
6   // Se declaran las variables para la cantidad, las denominaciones en las que
7   // se va a separar y una auxiliar para el proceso de separación
8   Definir cantidad, billetes100, monedas10, monedas1, resto Como Entero
9
10  // Se indica el propósito del algoritmo
11  Escribir "Se separará una cantidad indicada"
12  Escribir "en billetes de $100.00, monedas de $10.00 y monedas de $1.00"
13
14  // Se captura la cantidad a separar
15  Escribir Sin Saltar "Cantidad a separar:"
16  Leer cantidad
17
18  // Se separa la cantidad. Primero se obtienen los billetes de 100 dividiendo la cantidad
19  // entre 100 y truncando el cociente (eliminando la parte decimal)
20  billetes100 := trunc(cantidad / 100)
21  // Se obtiene lo que sobra después de quitar los billetes de 100
22  resto := cantidad MOD 100
23  // Se obtiene, de lo que sobró, las monedas de 10
24  monedas10 := trunc(resto / 10)
25  // Se obtiene lo que sobra después de quitar las monedas de 10, es decir las monedas de 1
26  monedas1 := resto MOD 10
27
28  // Se imprime la cantidad separada
29  Escribir "$", cantidad, ".00 se puede separar en ", billetes100, " billete(s) de $100.00,"
30  Escribir monedas10, " moneda(s) de $10.00 y ", monedas1, " moneda(s) de $1.00"
31 FinProceso

```

Sugerencia para ejercicio ².

2.9. Operadores relacionales

Los operadores relacionales sirven para formar expresiones lógicas, es decir, expresiones cuyo valor es VERDADERO o FALSO. A estos valores se les conoce como valores de verdad o de certeza. Por ejemplo, consideremos la expresión

$$a < b$$

²Incluir más denominaciones de billetes y monedas.

cuyo valor de verdad puede ser VERDADERO o FALSO dependiendo de los valores de a y de b, así pues, si $a=4$ y $b=10$ la expresión toma el valor VERDADERO mientras que si $a=5$ y $b=1$, toma el valor FALSO.

Otro ejemplo del uso de los operadores relacionales se muestra en la expresión

$$a \geq 'f'$$

donde se compara el contenido de una variable con respecto a un carácter. La variable a puede ser de tipo entero o caracter, en ambos se utiliza para la comparación el orden establecido en la tabla de códigos ASCII. Así pues, si $a='c'$, la expresión anterior es FALSO, en cambio si $a='r'$, es VERDADERO; si $a=20$, la expresión es FALSO pues el ASCII de 'f' es 102.

En el Cuadro 2.4 se muestran todos los operadores relacionales que se manejarán en este curso.

Cuadro 2.4: Operadores de relación o relacionales.

Símbolo matemático	Nombre	Símbolo en pseudocódigo
<	Menor que	<
≤	Menor o igual que	<=
>	Mayor que	>
≥	Mayor o igual que	>=
=	Igualdad lógico	=, ==
≠	Desigualdad lógica	<>, !=

Así pues, una expresión como $a \neq b$ se transforma en $a <> b$ y una expresión como $a \leq b$ se transforma en $a <= b$.

Una expresión como las anteriores se conoce como *expresión lógica simple*. Mediante operaciones entre expresiones lógicas simples se obtienen *expresiones lógicas compuestas*. En la sección siguiente presentaremos los operadores que utilizaremos para formar estas últimas.

2.10. Operadores lógicos

Una expresión lógica compuesta se forma utilizando operadores relacionales y los operadores lógicos AND, OR y NOT. Los operadores AND y OR requieren dos operandos lógicos, mientras que el operador NOT requiere un solo operando; dichos operandos generalmente se obtienen como resultado de evaluar expresiones lógicas simples, y dan como resultado otro valor de verdad. En PSeInt, el operador AND se representa con la letra **Y**, aunque también se puede usar el operador **&** y **&&**. El operador OR se representa con la letra **O**, o bien con **|** o **||**. Mientras que el operador de negación se representa con la palabra **NO** o con el operador **!**.

En el Cuadro 2.5 se muestran los valores de verdad para el operador Y aplicado a los valores x y y, donde F representa un valor FALSO y V, VERDADERO.

Cuadro 2.5: Tabla de verdad del operador lógico AND.

a	b	a Y b
V	V	V
V	F	F
F	V	F
F	F	F

Por ejemplo, considere la expresión compuesta

| (a>b) Y (c<=d)

Cuando a=10, b=3, c=1 y d=1, el término a>b toma el valor VERDADERO, además c<=d es VERDADERO, así que la expresión compuesta tiene valor VERDADERO.

En el Cuadro 2.6, se muestran los valores de verdad para el operador OR.

Por ejemplo, considere la expresión

| (a=1) O (b<>0)

Cuadro 2.6: Tabla de verdad del operador lógico OR.

a	b	a O b
V	V	V
V	F	V
F	V	V
F	F	F

Cuando $a:=7$ y $b:=15$, la expresión $a=1$ es FALSO, $b<>0$ es VERDADERO y la expresión compuesta toma el valor VERDADERO; por el contrario, cuando $a:=6$ y $b:=0$, el valor es FALSO.

En el Cuadro 2.7 se muestran los valores de verdad del operador NO.

Cuadro 2.7: Tabla de verdad del operador lógico NO.

x	NO x
V	F
F	V

Por ejemplo, la expresión

| NO($a<6$)

es VERDADERO cuando $a:=10$ y cuando $a:=1$, es FALSO

2.11. Leyes de DeMorgan

Las leyes de DeMorgan hacen posible escribir una expresión lógica compuesta de dos formas diferentes y, sin embargo, ser equivalentes. Esto debe tenerse en cuenta al comparar los algoritmos realizados por diferentes personas, pues es muy posible que difieran en apariencia pero que ambos estén escribiendo correctamente las instrucciones requeridas para resolver la situación planteada.

En el Cuadro 2.8 se muestran las leyes de DeMorgan, donde x_1 y x_2 representan a expresiones lógicas simples o compuestas.

Cuadro 2.8: Leyes de DeMorgan para la lógica.

$\text{NO}(x1 \text{ Y } x2) \equiv \text{NO } x1 \text{ O NO } x2$
$\text{NO}(x1 \text{ O } x2) \equiv \text{NO}x1 \text{ Y NO } x2$

Por ejemplo:

$$\text{NO}(x+y>0 \text{ Y } x \geq 0)$$

$$\equiv \text{NO}(x + y > 0) \text{ O NO}(x \geq 0)$$

$$\equiv (x + y \leq 0) \text{ Y } (x < 0)$$

Nótese que utilizamos paréntesis en cada expresión donde se aplica una operación NOT para indicar que ésta, será evaluada después de la expresión que aparece dentro de ellos; pero no utilizamos paréntesis para indicar que la suma debe realizarse antes de la comparación en la expresión $x+y>0$. Esto se debe a que el operador + tiene mayor precedencia que >.

En el Cuadro 2.9 se muestra un resumen de las precedencias de los operadores presentados hasta el momento.

Cuadro 2.9: Precedencia de los operadores.

Operadores				
()				
NOT	+	-	(monarios)	
* / MOD				
+	-	(binarios)		
<	<=	>	>=	
<> =				
Y, &, &&				
O, ,				

Con esto terminamos de presentar los elementos básicos de la programación en pseudocódigo.

Los algoritmos vistos hasta aquí constan de instrucciones que se ejecutan de manera secuencial, es decir, se inicia con la ejecución de la primera

instrucción, seguida de la ejecución de la segunda, etcétera; además, cada una de las instrucciones se ejecuta una única vez. Esta forma de programar es útil cuando se consideran problemas un tanto sencillos, pero, en general, es necesario tener mecanismos que permitan decidir entre la ejecución de una instrucción (o un conjunto de instrucciones) y otra, así como para permitir que se ejecuten en más de una ocasión. Esto es lo que permiten hacer las estructuras de control: elegir qué instrucciones se van a ejecutar cada vez.

Hay tres tipos de estructuras de control: la secuenciación, la selección y la repetición. Estos dos últimos temas se presentarán en los capítulos 3 y 4, respectivamente.

Capítulo 3 | Selección

La selección es una estructura de control que permite seleccionar el curso de ejecución de algunas instrucciones del programa en base al cumplimiento de alguna condición. Existen tres tipos de selección: simple, doble y múltiple. En las siguientes secciones se estudia cada una de ellas.

3.1. Selección simple

La selección simple es una estructura de control que permite decidir si se ejecuta o no, un conjunto de instrucciones. Dicha estructura se vale de una expresión lógica (simple o compuesta) llamada *condición* para decidir el flujo de acciones que se ejecutarán. El formato de la selección simple se muestra en la Figura 3.1.

```
Si condición Entonces
    Bloque de instrucciones a realizar cuando condición es verdadera
FinSi
```

Figura 3.1: Formato de la selección simple.

El flujo de acciones toma la siguiente línea: Si *condición* es VERDADERO, entonces se ejecuta el bloque de instrucciones dentro de la estructura, en caso contrario se ignora. El fin del bloque de instrucciones a ejecutar cuando condición es VERDADERO se especifica con la sentencia **FinSi**.

Se recomienda utilizar una sangría (también llamada indentación), para todas las instrucciones que estén en el bloque a ejecutar cuando la con-

dición sea verdadera. Es decir, hay que dejar la instrucción **Si** y **FinSi** a la misma altura. Esto es para mejorar la legibilidad de nuestro programa.

Leer el nombre y la edad del usuario. Saludarlo por su nombre y además, si el usuario tiene 18 años, mandarle un mensaje de felicitación. ←
EJERCICIO

A continuación, presentamos algunos ejemplos que ilustran el uso de esta estructura.

Ejemplo 3.1. Encontrar el menor de dos números.

Supóngase que se tienen dos valores almacenados cada uno en las variables *a* y *b*, respectivamente; y se desea compararlos de forma que se imprima el menor de ellos. Las instrucciones que se requieren para identificar si el menor es *a*, son las siguientes:

```
1 | Si a<b Entonces
2 |   escribir 'el menor es', a
3 | FinSi
```

Para identificar si el menor es *b*, es necesario modificar la condición y la instrucción de escritura. Los cambios realizados se muestran a continuación:

```
1 | Si a>b Entonces
2 |   escribir 'el menor es', b
3 | FinSi
```

También puede escribirse como

```
1 | Si b<a Entonces
2 |   escribir 'el menor es', b
3 | FinSi
```

En el Algoritmo 3.1 se muestran las instrucciones para resolver este problema considerándose además la posibilidad de que los valores dados sean iguales, incluyendo para esto, otra selección simple con la condición $a=b$.

Algoritmo 3.1: Menor de dos números.

```
1 // Nombre: MenorDeDos_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Se indica si dos números son iguales o cuál de ellos es menor
4
5 Proceso MenorDeDos
6   // Se declaran las variables para los dos números
7   Definir a, b Como Real
8
9   // Se indica el propósito del algoritmo
10  Escribir "Se indicará si dos números son iguales o cuál de ellos es el menor"
11
12  // Se capturan los dos números
13  Escribir Sin Saltar "Primer número:"
14  Leer a
15
16  Escribir Sin Saltar "Segundo número:"
17  Leer b
18
19  // Se comparan los números para saber si son iguales, si lo son, se indica
20  Si a = b Entonces
21    Escribir "Los números son iguales"
22  FinSi
23
24  // Se comparan los números para saber si el primero es menor que el segundo
25  Si a < b Entonces
26    Escribir "El menor es ", a
27  FinSi
28
29  // Se comparan los números para saber si el segundo es menor que el primero
30  Si b < a Entonces
31    Escribir "El menor es ", b
32  FinSi
33 FinProceso
```

Otra posible solución se muestra en el Algoritmo 3.2, en la cual se utiliza una variable auxiliar, `valorMenor` para guardar el valor menor de los valores considerados. Esta versión requiere de un par de selecciones simples anidadas, pues empieza considerando que `valorMenor` toma el valor a pero hace una comparación adicional y, si es necesario, lo cambia por b. Finalmente, escribe en pantalla el valor almacenado en `valorMenor`.

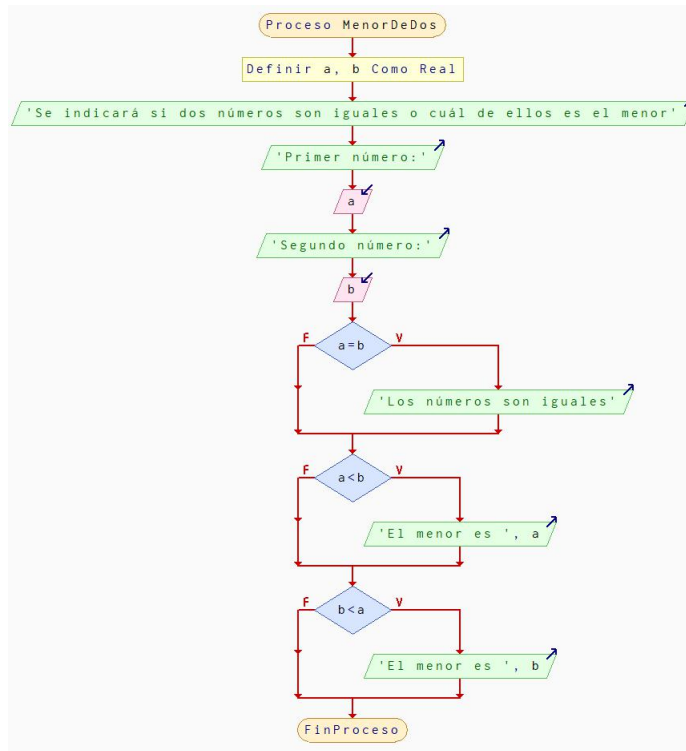


Figura 3.2: Diagrama de flujo.

Algoritmo 3.2: Menor de dos números usando una variable auxiliar.

```

1 // Nombre: MenorDeDos_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Se indica si dos números son iguales o cuál de ellos es menor
4
5 Proceso MenorDeDos
6   // Se declaran las variables para los dos números y una variable auxiliar
7   // para guardar el menor
8   Definir a, b, valorMenor Como Real
9
10  // Se indica el propósito del algoritmo
11  Escribir "Se indicará si dos números son iguales o cuál de ellos es el menor"
12
13  // Se capturan los dos números
14  Escribir Sin Saltar "Primer número:"
15  Leer a
16
17  Escribir Sin Saltar "Segundo número:"
18  Leer b
19
20  // Se comparan los números para saber si son iguales, si lo son, se indica

```

```

21  Si a = b Entonces
22      Escribir "Los números son iguales"
23  FinSi
24
25  // Si los números son distintos, se busca al menor
26  Si a <> b Entonces
27      // Se guarda en la variable auxiliar el primer número
28      valorMenor <- a
29      // Si el segundo número es menor que el almacenado como menor, se actualiza
30      // la variable auxiliar
31      Si b < valorMenor Entonces
32          valorMenor <- b
33      FinSi
34      // Se imprime el menor (almacenado en la variable auxiliar)
35      Escribir "El menor es ", valorMenor
36  FinSi
37  FinProceso

```

Ejemplo 3.2. Determinar si un número entero dado es par.

Almacenaremos el número en cuestión en la variable n . Para identificar si n es par utilizaremos el operador MOD ya que si $n \text{ MOD } 2$ da como resultado el valor 0, entonces la división de n entre 2 tiene residuo cero, lo cual significa que n es par; de aquí que si la condición $n \text{ MOD } 2 = 0$ es VERDADERO, n es par. Para revisar si se trata de un número impar, debe evaluarse la condición $n \text{ MOD } 2 \neq 0$. En el Algoritmo 3.3 se muestra el proceso completo.

Algoritmo 3.3: Determinar si un número entero dado es par.

```

1  // Nombre: Paridad.psc
2  // Autor: Irene iRodriguez y Edelmira iRodriguez
3  // Descripción:
4
5  Proceso Paridad
6      Definir n Como Entero
7
8      Escribir "Este programa lee un numero entero y determina si es par o impar"
9      Escribir Sin Saltar "n="
10     Leer n
11
12     Si n MOD 2 = 0 Entonces
13         Escribir "El numero ", n, " es par"
14     FinSi
15
16     Si n MOD 2 <> 0 entonces
17         Escribir "El numero ", n, " es impar"
18     FinSi
19  FinProceso

```

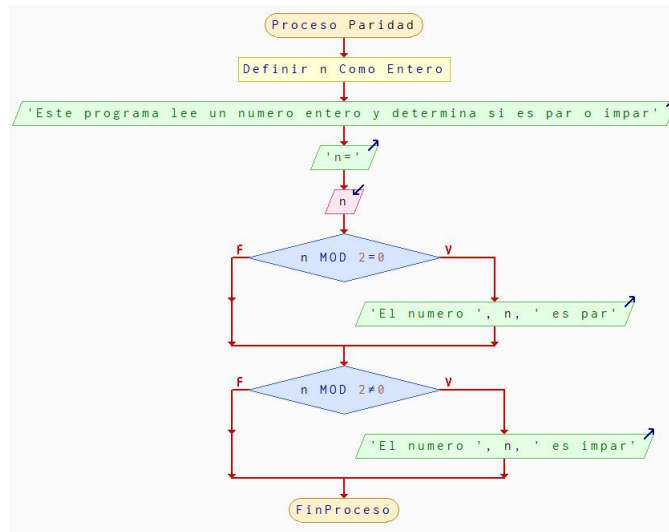


Figura 3.3: Diagrama de flujo del Algoritmo 3.3

Ejemplo 3.3. Contratación de un empleado.

Suponer que se aplican dos exámenes a un aspirante a trabajador y que solo si obtiene calificación mayor de 80 en ambos, se le contrata. Suponer que los resultados de las calificaciones están almacenados en las variables $c1$ y $c2$. Esta situación podría plantearse como se muestra en el Algoritmo 3.4, donde, una vez obtenidas las calificaciones $c1$ y $c2$, se procede a examinarlas para enviar el mensaje correspondiente según el caso. El problema especifica que solo en el caso de que ambas calificaciones sean mayores de 80 se contratará al aspirante; así pues, el operador AND es el que se adecúa a esta situación. También hay que mandar un mensaje en el caso en que no sea aceptado, es decir, la situación contraria que queda expresada en la segunda selección en el algoritmo. Nótese que es necesario el uso de paréntesis pues el operador NOT tiene mayor precedencia que el operador AND.

Algoritmo 3.4: Contratación de un empleado.

```

1 // Nombre: Contratacion.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso Contratacion
6   Definir c1, c2 Como Real
7
8   Escribir "Este programa lee dos calificaciones de un aspirante a un trabajo."
9   Escribir "Se le contratara solo si ambas calificaciones son mayores a 80"
10
11  Escribir Sin Saltar "Calificación 1="
12  Leer c1
13
14  Escribir Sin Saltar "Calificación 2="
15  Leer c2
16
17  Si c1>80 y c2>80 Entonces
18    Escribir "Contratado"
19  FinSi
20
21  Si ~(c1>80 y c2>80) Entonces
22    Escribir "NO contratado"
23  FinSi
24 FinProceso

```

Para plantear la situación en que no se contrata a la persona, puede utilizarse también la condición equivalente $c1 \leq 80$ O $c2 \leq 80$, lo cual se deduce de una de las leyes de DeMorgan ilustradas en el Cuadro 2.8, ya que

$$\begin{aligned}
 NOT(c1 > 80 \text{ AND } c2 > 80) &\equiv NOT(c1 > 80) \text{ OR } NOT(c2 > 80) \\
 &\equiv c1 \leq 80 \text{ OR } c2 \leq 80
 \end{aligned}$$

Obsérvese que en los últimos dos ejemplos se han utilizado dos selecciones simples cuyas condiciones son complementarias, es decir, cuando una es FALSO la otra es VERDADERO, y viceversa; con lo cual, aseguramos que solamente uno de los dos bloques será ejecutado. Esta situación puede plantearse de forma más compacta utilizando una selección doble, ésta será presentada en la siguiente sección.

3.2. Selección doble

La selección simple está limitada a especificar las acciones a realizar cuando la condición lógica es VERDADERO, pero no especifica qué hacer cuando el valor lógico de la condición es FALSO. La selección doble considera ambos casos, pues especifica las operaciones a realizar para cada uno de los valores de la condición lógica. La primera parte de la selección doble continúa siendo igual a la simple, es decir, si la condición lógica es VERDADERO, se realizan las operaciones especificadas por el primer bloque de instrucciones, pero si la condición es FALSO se realizan las instrucciones indicadas en el segundo bloque cuyo inicio se indica con las palabras **Sino**. El fin de la selección es determinada por la instrucción **FinSi**. El formato se ilustra en la Figura 3.4.

Si *condición* **Entonces**

Bloque de instrucciones a realizar cuando *condición* es verdadera

Sino

Bloque de instrucciones a realizar cuando *condición* es falsa

FinSi

Figura 3.4: Formato de la selección doble.

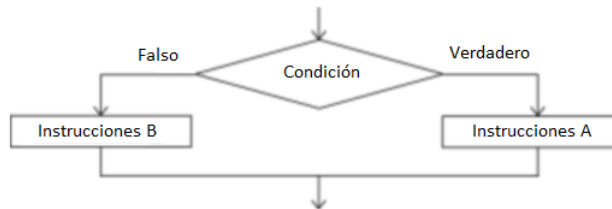


Figura 3.5: Formato de la selección doble.

Los siguientes ejemplos ilustran el uso de esta estructura de control.

Ejemplo 3.4. Encontrar el mayor de dos números dados.

Se tienen dos valores almacenados, *a* y *b*, y se deben comparar para identificar cuál de ellos es el mayor. En el Ejemplo 3.1 se analizó un proble-

ma similar. En esta sección se resolverá utilizando una selección doble tal como se muestra en el Algoritmo 3.5 y en el Algoritmo 3.6.

Algoritmo 3.5: Encontrar el mayor de dos números dados.

```

1 // Nombre: MayorDeDos_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso MayorDeDos
6   Definir a, b Como Real
7
8   Escribir "Este programa lee dos números reales y determina cual es el mayor"
9   Escribir Sin Saltar "Valor de a ="
10  Leer a
11
12  Escribir Sin Saltar "Valor de b ="
13  Leer b
14
15  Si a>b Entonces
16    Escribir "El mayor es ",a
17  SiNo
18    Escribir "El mayor es ",b
19  FinSi
20 FinProceso

```

Obsérvese que el caso de que la condición $a > b$ sea FALSO incluye la igualdad. Es decir, si a es igual a b se escribirá un letrero indicando que el mayor es b . Si nos interesa identificar el caso en que son iguales podemos utilizar el Algoritmo 3.6.

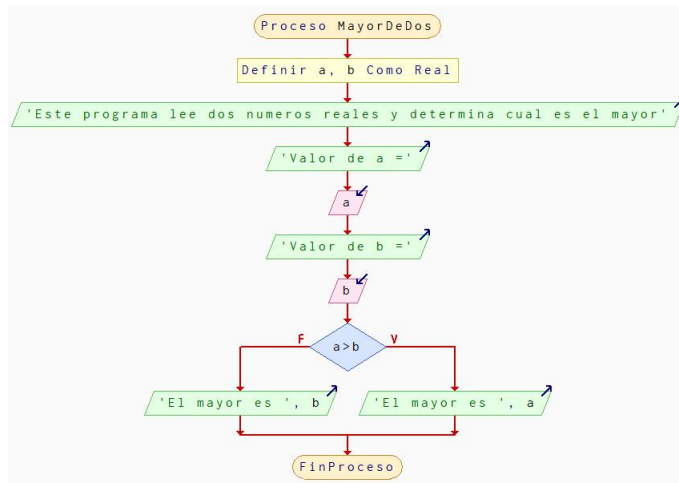


Figura 3.6: Diagrama de flujo del Algoritmo 3.5.

Algoritmo 3.6: Encontrar el mayor de dos números dados.

```
1 // Nombre: MayorDeDos_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso MayorDeDos_b
6   Definir a, b Como Real
7
8   Escribir "Este programa lee dos números reales y determina cuál es el mayor"
9
10  Escribir Sin Saltar "Valor de a="
11  Leer a
12
13  Escribir Sin Saltar "Valor de b="
14  Leer b
15
16  Si a>b Entonces
17    Escribir "El mayor es ",a
18  SiNo
19    Si a<b Entonces
20      Escribir "El mayor es ",b
21    SiNo
22      Escribir "Los números capturados son iguales"
23    FinSi
24  FinSi
25 FinProceso
```

Obsérvese que en caso de que la condición $a > b$ sea FALSO, aún es necesario considerar dos casos: $a < b$ y $a = b$. Por ello se utiliza de nuevo una selección doble. A esta forma de programar se le conoce como **anidamiento**. En general, es posible anidar tantas sentencias de selección (simple o doble) como sean necesarias.

Ejemplo 3.5. Encontrar el mayor de tres números dados.

En este problema se tienen 3 valores de entrada: a , b y c . Es necesario compararlos entre sí para obtener el mayor de ellos. Hay varias formas de hacer un algoritmo que resuelva este problema. Se presentarán cuatro de ellas. En el Algoritmo 3.7 se utilizan 10 estructuras de selección simple, tres de ellas están comparando cada uno de los valores para encontrar el mayor absoluto, una para verificar si ocurrió la igualdad y el resto para buscar el mayor cuando hay dos números iguales.

Algoritmo 3.7: Encontrar el mayor de tres números dados.

```
1 // Nombre: MayorDeTres_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso MayorDeTres_a
6   Definir a, b, c Como Real
7
8   Escribir "Este programa lee tres valores reales y determina cuál de ellos es el mayor"
9
10  Escribir Sin Saltar "Valor de a ="
11  Leer a
12
13  Escribir Sin Saltar "Valor de b ="
14  Leer b
15
16  Escribir Sin Saltar "Valor de c ="
17  Leer c
18
19  Si a>b y a>c Entonces
20    Escribir "El mayor es ",a
21  FinSi
22
23  Si b>a y b>c Entonces
24    Escribir "El mayor es ",b
25  FinSi
26
27  Si c>a y c>b Entonces
28    Escribir "El mayor es ",c
29  FinSi
30
31  Si c=a y c=b entonces
32    Escribir "Los números capturados son iguales"
33  FinSi
34
35  Si a=b y a>c Entonces
36    Escribir "El mayor es ",a
37  FinSi
38
39  Si a=b y a<c Entonces
40    Escribir "El mayor es ",c
41  FinSi
42
43  Si a=c y a>b Entonces
44    Escribir "El mayor es",a
45  FinSi
46
47  Si a=c y a<b Entonces
48    Escribir "El mayor es ",b
49  FinSi
50
51  Si b=c y b>a Entonces
52    Escribir "El mayor es ",b
53  FinSi
54
55  Si b=c y b<a Entonces
```

```

56     Escribir "El mayor es ",a
57     FinSi
58
59 FinProceso

```

En el Algoritmo 3.8 se muestra una versión con anidamiento de selecciones dobles. Se busca determinar primero el caso en que se da la igualdad de los tres valores dados. En caso de que no se cumpla la condición de igualdad se procede a buscar cuál es el mayor (incluyendo el caso en que haya dos iguales).

Algoritmo 3.8: Encontrar el mayor de tres números dados.

```

1  // Nombre: MayorDeTres_b.psc
2  // Autor:  Irene Rodríguez y Edelmira Rodríguez
3  // Descripción:
4
5  Proceso MayorDeTres_b
6      Definir a, b, c Como Real
7
8      Escribir "Este programa lee tres valores reales y determina cual de ellos es el mayor"
9
10     Escribir Sin Saltar "Valor de a ="
11     Leer a
12
13     Escribir Sin Saltar "Valor de b ="
14     Leer b
15
16     Escribir Sin Saltar "Valor de c ="
17     Leer c
18
19     Si a=b y a=c Entonces
20         Escribir "Los números capturados son iguales"
21     SiNo
22         Si a>=b y a>=c Entonces
23             Escribir "El mayor es ",a
24         SiNo
25             Si b>=c Entonces
26                 Escribir "El mayor es ",b
27             SiNo
28                 Escribir "El mayor es ",c
29         FinSi
30     FinSi
31 FinProceso

```

El Algoritmo 3.9 utiliza una variable auxiliar, mayor3, inicializada con el valor de uno de los valores considerados, en este caso a, después procede a hacer las comparaciones de mayor con los otros dos valores, b y c, por separado, y, si es necesario, modifica el valor de mayor3.

Algoritmo 3.9: Encontrar el mayor de tres números dados.

```
1 // Nombre: MayorDeTres_c.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso MayorDeTres_c
6   Definir a, b, c, mayor3 Como Real
7
8   Escribir "Este programa lee tres valores reales y determina cual de ellos es el mayor"
9
10  Escribir Sin Saltar "Valor de a ="
11  Leer a
12
13  Escribir Sin Saltar "Valor de b ="
14  Leer b
15
16  Escribir Sin Saltar "Valor de c ="
17  Leer c
18
19  Si a=b y b=c Entonces
20    Escribir "Los números capturados son iguales"
21  SiNo
22    mayor3 := a
23    Si b>mayor3 Entonces
24      mayor3 := b
25    FinSi
26
27    Si c>mayor3 Entonces
28      mayor3 := c
29    FinSi
30
31    Escribir "El mayor es ", mayor3
32  FinSi
33
34 FinProceso
```

Ejemplo 3.6. Contratación de un empleado (uso de AND).

Se aplican dos exámenes a un trabajador. Solo si obtiene más de 80 en ambos, se contrata. Considere el problema del Ejemplo 3.3 con la variante de que esta vez se utiliza selección doble. En el Algoritmo 3.10 se presenta esta nueva versión.

Algoritmo 3.10: Contratación de un empleado (usando AND).

```
1 // Nombre: Contratacion2.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Este programa lee dos calificaciones y manda un mensaje de contratado
4 // cuando ambas son mayores de 80.
5
6 Proceso Contratacion2
7   Definir c1, c2 Como Real
8
9   Escribir "Este programa lee dos calificaciones de un aspirante a un trabajo."
10  Escribir "Se le contratara solo si ambas calificaciones son mayores a 80"
11
12  Escribir Sin Saltar "Calificacion 1="
13  Leer c1
14
15  Escribir Sin Saltar "Calificacion 2="
16  Leer c2
17
18  Si c1>80 y c2>80 Entonces
19    Escribir "Contratado"
20  SiNo
21    Escribir "NO contratado"
22  FinSi
23 FinProceso
```

Ejemplo 3.7. Contratación de un empleado (uso de OR).

Se aplican dos exámenes a un aspirante a trabajador y sólo si obtiene calificación mayor de 80 en al menos uno de ellos será contratado. En este caso el operador lógico requerido es el operador OR, tal como se muestra en el Algoritmo 3.11.

Algoritmo 3.11: Contratación de un empleado (usando OR).

```
1 // Nombre: Contratacion3_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Este programa lee dos calificaciones y manda un mensaje de contratado
4 // cuando al menos una es mayor de 80.
5
6 Proceso Contratacion3
7   Definir c1, c2 Como Real
8
9   Escribir "Este programa lee dos calificaciones de un aspirante a un trabajo."
10  Escribir "Se le contratara solo si ambas calificaciones son mayores a 80"
11
12  Escribir Sin Saltar "Calificacion 1="
13  Leer c1
14
15  Escribir Sin Saltar "Calificacion 2="
16  Leer c2
17
18  Si c1>80 o c2>80 Entonces
19    Escribir "Contratado"
20  SiNo
21    Escribir "NO contratado"
22  FinSi
23 FinProceso
```

En el Algoritmo 3.12, se muestra una segunda implementación utilizando una variante de la selección doble donde se invierte el orden de los mensajes enviados al usuario.

Algoritmo 3.12: Contratación de un empleado.

```
1 // Nombre: Contratacion3_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Este programa lee dos calificaciones y manda un mensaje de contratado
4 // cuando ambas son mayores de 80.
5
6 Proceso Contratacion3_b
7   Definir c1, c2 Como Real
8
9   Escribir "Este programa lee dos calificaciones de un aspirante a un trabajo."
10  Escribir "Se le contratara solo si ambas calificaciones son mayores a 80"
11
12  Escribir Sin Saltar "Calificación 1="
13  Leer c1
14
15  Escribir Sin Saltar "Calificación 2="
16  Leer c2
17
18  Si c1<=80 AND c2<=80 Entonces
19    Escribir "No contratado"
20  SiNo
21    Escribir "Contratado"
22  FinSi
23 FinProceso
```

Sugerencia de tarea.¹

¹ Si el aspirante tiene mas de 80 en ambos exámenes, se le contrata. Si obtiene menos de 80 en uno de los exámenes pero mas de 90 en el otro, pasa a una segunda ronda de selección. En otro caso, no se le contrata.

Ejemplo 3.8. Saludando al usuario por su nombre.

En este ejercicio, vamos a solicitar y leer el nombre del usuario, para mandarle un saludo personalizado. Consideramos que dejar en blanco la respuesta a esta pregunta será un caso erróneo, por lo que vamos a enviar el mensaje de saludo solo si la longitud de la respuesta no es cero. En el Algoritmo 3.13, se implementa este funcionamiento.

Algoritmo 3.13: Saludando al usuario por su nombre.

```
1 // Nombre: Saludo_usuario_a.psc
2 // Autor: Edelmira
3 // Fecha: 15-agosto-2024
4 // Descripción: Este programa envia un saludo personalizado.
5 Proceso Saludando1
6     Definir nombre como Caracter
7
8     Escribir "Este programa envia un saludo personalizado"
9     Escribir "Teclea tu nombre " Sin Saltar
10    Leer nombre
11
12    Si longitud(nombre)>0 Entonces
13        Escribir "Hola, ", nombre
14        Escribir "Esta es tu bienvenida al mundo de la programación en PSeInt"
15    SiNo
16        Escribir "ERROR: Este campo no puede quedar vacío"
17    FinSi
18 FinProceso
```

Hay una variante que se puede utilizar para prevenir que la respuesta del usuario a la pregunta del nombre sea vacía, y es comparar con la cadena vacía. La condición que puede usarse entonces es

```
Si nombre <> "" Entonces
    Escribir "Hola, ", nombre, ", esta es tu bienvenida al mundo de la programacion en PSeInt"
SiNo
    Escribir "ERROR: Este campo no puede quedar vacio"
FinSi
```

Ejemplo 3.9. Enviar un mensaje personalizado al usuario, dados su nombre y edad. Se cuida que el valor en el nombre sea no vacío y que el valor de la edad esté en el rango [5,100]. Como detalle adicional, si la edad es 18, mandaremos un mensaje felicitando al usuario por haber llegado a la mayoría de edad.

Algoritmo 3.14: Saludando al usuario por su nombre y edad.

```

1 // Nombre: Saludo_usuario_b.psc
2 // Autor: Edelmira
3 // Fecha: 15-agosto-2024
4 // Descripción: Este programa envía un saludo personalizado, dados el nombre y la edad
5 Proceso Saludando2
6   Definir nombre como Caracter
7   Definir edad como Entero
8
9   Escribir "Este programa envía un saludo personalizado"
10  Escribir "Se te pedirá el nombre (no debe quedar vacío)"
11  Escribir "y tu edad, se espera un valor en [5,100]"
12
13  Escribir ""
14  Escribir "Teclea tu nombre " Sin Saltar
15  Leer nombre
16
17  Si longitud(nombre)>0 Entonces
18    Escribir "Hola,", nombre,", esta es tu bienvenida al mundo de la programación en PSeInt"
19    Escribir "Teclea tu edad " Sin Saltar
20    Leer edad
21    Si edad>=5 y edad<=100 Entonces // puedes modificar este rango
22      Escribir "Tu edad es ", edad
23      Si edad=18 Entonces
24        Escribir "FELICIDADES!! Ahora eres mayor de edad :) "
25      FinSi
26    Sino
27      Escribir "ERROR: Se espera un valor entero en el rango [5,100]"
28    FinSi
29  SiNo
30    Escribir "ERROR: Este campo no puede quedar vacío"
31  FinSi
32 FinProceso

```

Cabe señalar que si se deja en blanco la respuesta a una pregunta con valor numérico, PSeInt la inicializará con el valor cero. La única alternativa para evitar que no quede en blanco, es declarar la respuesta como una cadena y después convertirla a un número. En el siguiente segmento de código se ilustra el procedimiento

Definir entrada **Como** Cadena

```
Definir edad Como Entero

Escribir "Ingresa un número entero:"
Leer entrada

Si entrada <> "" Entonces
    edad <- ConvertirANumero(entrada)
    Escribir "Número ingresado: ", edad
sino
    Escribir "ERROR: Este campo no puede quedar vacío"
FinSi
```

Aunque si el usuario captura un valor no válido para hacer la conversión a entero, por ejemplo "hola", se lanzará un error. En el caso de captura de números reales, es la misma idea. Cuando veamos el tema de funciones, se verá una alternativa para solucionar este problema.

Ejemplo 3.10. Enviar un mensaje personalizado al usuario, dados su nombre, edad y peso. Se cuida que el valor en el nombre sea no vacío, que el valor de la edad esté en el rango [5,100] y que el rango de valores aceptados para el peso sea [30,130]. Cuando la edad es 18, mandaremos un mensaje felicitando al usuario por haber llegado a la mayoría de edad.

Algoritmo 3.15: Saludando al usuario por su nombre, edad y peso.

```

1 // Nombre: Saludo_usuario_c.psc
2 // Autor: Edelmira
3 // Fecha: 15-agosto-2024
4 // Descripción: Este programa envía un saludo personalizado, dados el nombre, la edad
5 // y el peso. El objetivo es cuidar el rango de valores de cada variable.
6 Proceso Saludando3
7   Definir nombre como Caracter
8   Definir edad como Entero
9   Definir peso como Real
10
11   Escribir "Este programa envía un saludo personalizado"
12   Escribir "Se te pedirá el nombre (no debe quedar vacío)"
13   Escribir "se te pedirá tu edad, se espera un valor en [5,100]"
14   Escribir "se te pedirá tu peso, se espera un valor en [30,130]"
15
16   Escribir ""
17
18   Escribir "Teclea tu nombre " Sin Saltar
19   Leer nombre
20
21   Si longitud(nombre)>0 Entonces
22     Escribir "Hola,", nombre
23     Escribir "Esta es tu bienvenida al mundo de la programación en PSeInt "
24     Escribir "Teclea tu edad " Sin Saltar
25     Leer edad
26     Si edad>=5 y edad<=100 Entonces
27       Escribir "Captura tu peso (kgs) " Sin Saltar
28       Leer peso
29       Si peso>=30 y edad<=130 Entonces
30         Escribir nombre, "tu edad es de ",edad, " años y pesas ", peso, " kgs"
31         Si edad=18 Entonces
32           Escribir "FELICIDADES!! Ya eres mayor de edad :) "
33         FinSi
34       Sino
35         Escribir "ERROR: Se espera un valor en el rango [30,130]"
36       FinSi
37     Sino
38       Escribir "ERROR: Se espera un valor entero en el rango [5,100]"
39     FinSi
40   SiNo
41     Escribir "ERROR: Este campo no puede quedar vacío"
42   FinSi
43 FinProceso

```

Ejemplo 3.11. Calcular el Índice de Masa Corporal (IMC) del usuario, usando la fórmula

$$IMC = \frac{peso}{estatura^2}$$

y enviar un mensaje relacionado con su peso, utilizando la siguiente tabla

IMC	Mensaje
<18.5	Estás por debajo del rango de peso normal para tu estatura
[18.5,25]	Estás en el rango de peso normal para tu estatura
>25	Estás por encima del rango de peso normal para tu estatura

Al ejemplo 3.10, hay que agregarle la lectura de la estatura del usuario, cuidando que el valor esté en el rango [1.5, 2] mts. Una vez capturado este valor, se calcula el valor del IMC con la fórmula indicada y se manda el mensaje correspondiente según el resultado. En el Algoritmo 3.16 se muestra una posible solución a este problema.

Algoritmo 3.16: Índice de masa corporal.

```

1 // Nombre: IMC.psc
2 // Autor: Edelmira
3 // Fecha: 21-agosto-2024
4 // Descripción: Este programa calcula el IMC del usuario
5 // y manda un mensaje según el valor de este dato
6 Proceso Calcular_IMC
7   Definir nombre como Caracter
8   Definir edad como Entero
9   Definir peso, estatura, imc como Real
10
11   Escribir "Este programa envia un saludo personalizado"
12   Escribir "Se te pide el nombre (no debe quedar vacío)"
13   Escribir "Se te pide tu edad, se espera un valor en [18,100] años"
14   Escribir "Se te pide tu peso, se espera un valor en [30,130] kgs"
15   Escribir "Se te pide tu estatura, se espera un valor en [1.5,2] mts"
16
17   Escribir ""
18   Escribir "Teclea tu nombre " Sin Saltar
19   Leer nombre
20   Si longitud(nombre)>0 Entonces
21     Escribir "Teclea tu edad " Sin Saltar
22     leer edad
23     Si edad>=18 y edad<=100 Entonces
24       Escribir "Captura tu peso (kgs) " Sin Saltar
25       Leer peso
26       Si peso>=30 y edad<=130 Entonces
27         Escribir "Captura tu estatura (mts) " Sin Saltar

```

```

28      Leer estatura
29      Si estatura>=1.5 y estatura<=2.0 Entonces
30          Limpiar Pantalla
31          Escribir "Hola,", nombre,", tienes ",edad, " años, "
32          Escribir "pesas ", peso, " kgs y mides ",estatura, " mts"
33          imc := peso / estatura ^2
34          Si imc<18.5 Entonces
35              Escribir "Estás por DEBAJO del rango de peso normal para tu estatura"
36          SiNo
37              Si imc<25 Entonces
38                  Escribir "Estás en el rango de peso normal para tu estatura"
39              SiNo
40                  Escribir "Estás por ENCIMA del rango de peso normal para tu estatura"
41              FinSi
42          FinSi
43          SiNo
44              Escribir "ERROR: Se espera un valor en el rango [1.5, 2.0]"
45          FinSi
46          Sino
47              Escribir "ERROR: Se espera un valor en el rango [30, 130]"
48          FinSi
49          Sino
50              Escribir "ERROR: Se espera un valor entero en el rango [18, 100]"
51          FinSi
52          SiNo
53              Escribir "ERROR: Este campo no puede quedar vacío"
54          FinSi
55      FinProceso

```

Sugerencia de ejercicio.²

²Calcular el IMC considerando una tabla con más categorías (bajo peso extremo, obesidad grado 1, etc.)

Ejemplo 3.12. Determinar si un año dado es bisiesto.

El usuario proporciona el valor del año, cuidaremos que esté en el rango [1800, 2100]. Un año es bisiesto si se cumple lo siguiente

- Es múltiplo de 4 y no es múltiplo de 100, o bien
- es múltiplo de 400

Algoritmo 3.17: Día de la semana (v2).

```
1 // Nombre: Contratacion.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Este programa lee un valor correspondiente a un año en el rango [1800,2100]
4 // y determina si se trata de un año bisiesto o no.
5
6 Proceso Bisiesto
7   Definir year, mes, numDias Como Entero
8
9   Escribir "Captura el año [1800, 2100] " sin saltar
10  Leer year
11
12  Si year>=1800 y year<=2100 Entonces
13    si year mod 4=0 y year mod 100<>0 o year mod 400=0 Entonces
14      Escribir year " es año bisiesto"
15    SiNo
16      Escribir year " no es año bisiesto"
17    FinSi
18  SiNo
19    Escribir "Se espera un valor en el rango [1800,2100] "
20  FinSi
21 FinProceso
```

Ejemplo 3.13. Dado un número entero entre 1 y 7 indíquese a que día de la semana corresponde.

Considerar que los días de la semana se numeran a partir del lunes. Se presentan dos versiones de un algoritmo que plantea esta situación. En el Algoritmo 3.18, se utilizan varias selecciones simples cada una de las cuales considera un posible caso. Así, se considera que la variable día, donde se almacenó el valor leído, puede tomar cualesquiera de los valores 1, 2, ..., 7 y se considera, adicionalmente, el caso en que día tomó un valor diferente a los especificados. Este último caso es necesario pues aún cuando al usuario se le solicita un valor entre 1 y 7, es posible que proporcione un valor que no está en el rango mencionado (por error o de forma intencional), en este caso, hay que enviar un mensaje al usuario indicando cuál es la situación.

Algoritmo 3.18: Día de la semana (v1).

```
1 // Nombre: DiaSemana_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso DiaSemana
6   Definir dia Como Entero
7
8   Escribir Sin Saltar "Escribe un numero entero entre 1 y 7"
9   Leer dia
10
11  Si dia=1 Entonces
12    Escribir "Lunes"
13  FinSi
14
15  Si dia=2 Entonces
16    Escribir "Martes"
17  FinSi
18
19  Si dia=3 Entonces
20    Escribir "Miércoles"
21  FinSi
22
23  Si dia=4 Entonces
24    Escribir "Jueves"
25  FinSi
26
27  Si dia=5 Entonces
28    Escribir "Viernes"
29  FinSi
30
31  Si dia=6 Entonces
32    Escribir "Sábado"
33  FinSi
34
35  Si dia=7 Entonces
36    Escribir "Domingo"
37  FinSi
38
39  Si dia<1 o dia>7 Entonces
40    Escribir "Valor fuera de rango"
41  FinSi
42 FinProceso
```

En el Algoritmo 3.19, se plantea la situación de manera diferente: se consideran varias estructuras de selección dobles anidadas, cada una considerando una posibilidad para el valor de la variable dia. En la selección más interna se considera la situación en que el valor dado por el usuario no ha sido ninguno de los considerados en el rango deseado, y por lo tanto, se enviará un mensaje al usuario mencionando el error cometido.

Algoritmo 3.19: Día de la semana (v2).

```

1 // Nombre: DiaSemana_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso DiaSemana_b
6   Definir dia Como Entero
7
8   Escribir Sin Saltar "Escribe un numero entero entre 1 y 7"
9   Leer dia
10
11   Si dia=1 Entonces
12     Escribir "Lunes"
13   SiNo
14     Si dia=2 Entonces
15       Escribir "Martes"
16     SiNo
17       Si dia==3 Entonces
18         Escribir "Miércoles"
19       SiNo
20         Si dia=4 entonces
21           escribir "Jueves"
22         SiNo
23           Si dia=5 Entonces
24             Escribir "Viernes"
25           SiNo
26             Si dia=6 Entonces
27               Escribir "Sábado"
28             SiNo
29               Si dia=7 Entonces
30                 Escribir "Domingo"
31               SiNo
32                 Escribir "Valor fuera de rango"
33             FinSi
34           FinSi
35         FinSi
36       FinSi
37     FinSi
38   FinSi
39 FinSi
40
41 FinProceso

```

Nótese la diferencia entre ambas versiones del algoritmo: en la ejecución del Algoritmo 3.18, se deben revisar cada una de las condiciones para ver si se cumplen de forma individual así que se realizan 8 comparaciones. En cambio, en el Algoritmo 3.19 se revisan las diferentes condiciones hasta que una de ellas sea verdadera de tal forma que se realizan *a lo más* 7 comparaciones.

EJERCICIO:

1. Escribir el nombre del mes según corresponde, dado un número entero en el rango [1,12].
2. Determinar el número de días del mes en un año dado.

Existe una estructura que simplifica mucho el uso de la selección simple y doble cuando se tienen casos como el del último ejemplo, nos referimos a la selección múltiple presentada en la siguiente sección.

3.3. Selección múltiple

Cuando se tienen que revisar múltiples comparaciones de equivalencia como en el Ejemplo 3.13, es mejor utilizar la selección múltiple pues, como mencionamos, simplifica el procedimiento. El formato a utilizar se muestra en la Figura 3.7.

Segun *variable* **Hacer**

número1: Bloque de instrucciones a realizar cuando *variable*=*número1*

número2: Bloque de instrucciones a realizar cuando *variable*=*número2*

...

De otro modo: Bloque de instrucciones a realizar cuando *variable*
toma un valor diferente a los enlistados

FinSegun

Figura 3.7: Formato de la selección múltiple.

Donde *variable* puede ser de tipo entero únicamente. Se muestra un listado con los posibles valores de *variable* (*número1*, *número2*, ...) acompañados por el bloque de instrucciones que se realizarán en cada caso. Así, si *variable* toma el valor *número1* se ejecutará el bloque de instrucciones que se encuentra indicado, y así sucesivamente. Cuando *variable* toma un valor diferente a los enlistados se ejecutará el bloque de instrucciones por defecto. Cabe aclararse que esta última sección de la estructura es opcional, es decir, puede omitirse, en cuyo caso no se ejecutará acción alguna cuando *variable* toma un valor no considerado en la lista.

La implementación del Algoritmo 3.18 utilizando selección múltiple se presenta en el Algoritmo 3.20.

Algoritmo 3.20: Día de la semana (v3).

```
1 // Nombre: DiaSemana_c.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso DiaSemana_c
6   Definir dia Como Entero
7
8   Escribir Sin Saltar "Escribe un numero entero entre 1 y 7"
9   Leer dia
10
11   Segun dia
12     1: Escribir "Lunes"
13     2: Escribir "Martes"
14     3: Escribir "Miércoles"
15     4: Escribir "Jueves"
16     5: Escribir "Viernes"
17     6: Escribir "Sábado"
18     7: Escribir "Domingo"
19   De Otro Modo:
20     Escribir "Valor fuera de rango"
21   FinSegun
22 FinProceso
```

Cuando queremos mostrarle al usuario una serie de opciones para que elija una de ellas, pueden presentarse en un listado o menú de opciones. Tal como se ilustra en el Ejemplo 3.14.

Ejemplo 3.14. Menú de opciones.

Presentar al usuario un listado de nombres de estados de la República Mexicana, codificados por un número entero, y pedirle que elija su estado de nacimiento utilizando el número que le corresponde en la lista. Una vez que selecciona su estado de nacimiento, mandarle un mensaje indicando el gentilicio apropiado. Aunque lo ideal es mostrar la lista que incluya a los 31 estados de nuestro país, por cuestiones de espacio, se redujo a cinco elementos en el Algoritmo 3.21

Algoritmo 3.21: Menu de opciones.

```
1 // Nombre: Menu1.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4 Proceso Menu
5
6   Definir estado como Entero
7
8   Escribir "Menu de estados de nacimiento disponible"
9   Escribir "1) Sonora"
10  Escribir "2) Sinaloa"
11  Escribir "3) Chihuahua"
12  Escribir "4) Baja California"
13  Escribir "5) Jalisco"
14  Escribir "En que estado naciste? " sin saltar
15  Leer estado
16
17  Segun estado hacer
18    1: Escribir "Elegiste Sonora, eres sonorense"
19    2: Escribir "Elegiste Sinaloa, eres sinaloense"
20    3: Escribir "Elegiste Chihuahua, eres chihuahuense"
21    4: Escribir "Elegiste Baja California, eres bajacaliforniano"
22    5: Escribir "Elegiste Jalisco, eres jalisciense"
23
24  De Otro Modo:
25    Escribir "ERROR: Se espera un valor en el rango [1,5]"
26  FinSegun
27 FinProceso
```

Ejemplo 3.15. Determinar el número de días del mes en un año dado.

El usuario proporciona dos valores: el año y el mes en cuestión. Para que pueda indicar el mes, le presentamos un menú con los nombres de los meses codificados por un número entero entre 1 y 12.

Hay que determinar si el año es bisiesto, para ello se evalúa que el año sea múltiplo de 4 pero no de 100 o bien, que sea múltiplo de 400. Es decir, se requiere de las siguientes instrucciones:

```
Definir year como entero
Escribir "Teclea el año a evaluar si es bisiesto " sin saltar
Leer year

si year mod 4 = 0 y year mod 100 <> 0 o year mod 400 = 0 entonces
    Escribir "El año ", year, " es bisiesto"
FinSi
```

Ejemplo 3.16. Lanzar un par de dados y obtener la suma.

En algunos programas, sobre todo cuando se trata de juegos, se requiere simular el lanzamiento de un dado. Para simular el lanzamiento de un dado, se puede utilizar la función **Aleatorio(minimo,maximo)**, que regresa un valor entero en el rango [minimo,maximo].

En este ejemplo, lanzaremos dos dados, por lo que se utilizará la función Aleatorio() dos veces, una por cada dado y se sumarán los resultados obtenidos. Las instrucciones a utilizar son

```
dado1 := aleatorio(1,6)
dado2 := aleatorio(1,6)
suma_datos := dado1 + dado2
```

Obsérvese que no es correcto utilizar la función aleatorio(1,12) pues, aunque se estarían considerando los valores 1, 2, ..., 12, el 1 no es un valor que pueda obtenerse al lanzar dos dados. Es posible utilizar la expresión aleatorio(2,12) con lo cual estaríamos obteniendo valores del 2 al 12, pero dicha expresión no simularía el lanzamiento de dos dados, puesto que la probabilidad de ocurrencia de cada resultado se ve afectada. En el Algoritmo 3.22, se muestra el algoritmo utilizado para este fin.

Algoritmo 3.22: Lanzando dados.

```
1 // Nombre: LanzandoDados.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4 Proceso TirandoDados
5     Definir dado1,dado2,sumaDados como Entero
6
7     dado1 := Aleatorio(1,6)
8     dado2 := Aleatorio(1,6)
9
10    Escribir "Este programa lanza dos dados e imprime la suma de ellos"
11    Escribir "dado1= ", dado1
12    Escribir "dado2= ", dado2
13
14    sumaDados := dado1+dado2
15
16    Escribir "La suma de los dados es ",sumaDados
17 FinProceso
```

Hay otra función para generar números aleatorios en PSeInt: la función **azar(limiteSup)**. Esta función regresa un número aleatorio en el rango [0,limiteSuperior). NOTA: No incluye el valor dado como límite superior del rango a utilizar. En caso de utilizarse para este ejemplo, las instrucciones quedarían

```
dado1 := azar(6)+1 // para trasladar el rango de valores a [1,6]
dado2 := azar(6)+1
suma_dados := dado1 + dado2
```

Ejemplo 3.17. El juego del 7.

En este juego participa un jugador y se requieren dos dados. El jugador tira ambos dados y suma las caras superiores. Previamente, se hace una apuesta sobre lo que será la suma de los dados (abajo de 7, 7 ó arriba de 7). En caso de que haya apostado a que la suma sería abajo del siete y acertó, gana la suma apostada. Lo mismo cuando apostó arriba del 7. En caso de que haya elegido el 7 y acierta, gana el triple de la apuesta. Obsérvese que no importan las combinaciones de números obtenidos, sino la suma de los mismos. En caso de no acertar, pierde la cantidad apostada. Obviamente, las probabilidades siempre están en contra del jugador (sobre todo si elige apostar al 7).

Es necesario considerar las variables que representen la apuesta, la selección del usuario, los dados y la suma de los dados. Se presentan tres versiones de implementación del juego. De nuevo, es posible desarrollar versiones distintas que sean igualmente válidas. En el Algoritmo 3.23 se presenta una de las versiones consideradas.

Algoritmo 3.23: Juego El 7.

```

1 // Nombre: JuegoEl7_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso JuegoEl7_a
6   Definir jugador, dado1, dado2, suma_dados Como Entero
7   Definir apuesta, capital Como Real
8   Escribir "Este programa implementa el juego del 7, se acepta una apuesta"
9
10  Escribir Sin Saltar "Capital ="
11  Leer capital
12
13  Escribir Sin Saltar "Cuánto apuestas? $"
14  Leer apuesta
15
16  si apuesta <= capital entonces
17    // selección del usuario
18    escribir "Teclea el número que corresponde a tu selección "
19    escribir "1) Abajo del 7"
20    escribir "2) 7"
21    escribir "3) Arriba del 7"
22    leer jugador
23
24    // simulando los dados
25    dado1 := Azar(6)+1
26    dado2 := Azar(6)+1
27    suma_dados := dado1 + dado2
28
29    Escribir "Los dados cayeron en ", dado1, " y ", dado2, " suma = ", suma_dados
30
31    Si jugador=1 y suma_dados<7 Entonces
32      Escribir "Ganaste , tu capital es ahora de $", capital + apuesta
33    SiNo
34      Si jugador=2 y suma_dados=7 Entonces
35        Escribir "Ganaste , tu capital es ahora de $", capital + 3*apuesta
36      SiNo
37        Si jugador=3 y suma_dados>7 Entonces
38          Escribir "Ganaste , tu capital es ahora de $", capital + apuesta
39        SiNo
40          Escribir "Perdiste , tu capital es ahora de $", capital - apuesta
41        FinSi
42      FinSi
43    FinSi
44    SiNo
45      Escribir "No puedes apostar mas de lo que tienes..."
46    FinSi
47 FinProceso

```

En el Algoritmo 3.24, se agrupan los casos en que el usuario gana, pues en ambos casos se realiza la misma operación. Además, se eliminan dos variables, dado1 y dado2, al observar que sólo se utilizan para obtener el valor suma_dados.

Algoritmo 3.24: Juego El 7.

```

1 // Nombre: JuegoEl7_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso JuegoEl7_b
6   Definir jugador, dado1, dado2, suma_dados Como Entero
7   Definir apuesta, capital Como Real
8
9   Escribir "Este programa implementa el juego del 7, se acepta una apuesta"
10
11   Escribir Sin Saltar "Capital ="
12   Leer capital
13
14   Escribir Sin Saltar "Cuánto apuestas? $"
15   Leer apuesta
16
17   si apuesta <= capital entonces
18     // selección del usuario
19     escribir "Teclea el número que corresponde a tu selección "
20     escribir "1) Abajo del 7"
21     escribir "2) 7"
22     escribir "3) Arriba del 7"
23     leer jugador
24
25     // simulando los dados
26     dado1 := Azar(6)+1
27     dado2 := Azar(6)+1
28     suma_dados := dado1 + dado2
29
30     Escribir "Los dados cayeron en ", dado1, " y ", dado2, " suma = ", suma_dados
31
32     si jugador=2 y suma_dados=7 Entonces
33       Escribir "Ganaste , tu capital es ahora de $" , capital + 3*apuesta
34     SiNo
35       Si (jugador=1 y suma_dados<7) o (jugador=3 y suma_dados>7) Entonces
36         Escribir "Ganaste , tu capital es ahora de $" , capital + apuesta
37       SiNo
38         Escribir "Perdiste , tu capital es ahora de $" , capital - apuesta
39       FinSi
40     FinSi
41   SiNo
42     Escribir "No puedes apostar mas de lo que tienes..."
43   FinSi
44 FinProceso

```

En el Algoritmo 3.25 se utiliza una selección múltiple que considera todas las posibilidades para la opción del usuario y en cada caso se determina quien gana utilizando la selección doble.

Algoritmo 3.25: Juego El 7.

```

1 // Nombre: JuegoEl7_c.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez

```

```

3 // Descripción:
4
5 Proceso JuegoEl7_c
6   Definir jugador, dado1, dado2, suma_dados Como Entero
7   Definir apuesta, capital Como Real
8
9   Escribir "Este programa implementa el juego del 7, se acepta una apuesta"
10
11  Escribir Sin Saltar "Capital ="
12  Leer capital
13
14  Escribir Sin Saltar "Cuanto apuestas? $"
15  Leer apuesta
16
17  Si apuesta <= capital entonces
18    // seleccion del usuario
19    escribir "Teclea el número que corresponde a tu selección "
20    escribir "1) Abajo del 7"
21    escribir "2) 7"
22    escribir "3) Arriba del 7"
23    leer jugador
24
25    // simulando los dados
26    dado1 := Azar(6)+1
27    dado2 := Azar(6)+1
28    suma_dados := dado1 + dado2
29
30    Escribir "Los dados cayeron en ", dado1, " y ", dado2," suma = ",suma_dados
31
32  Segun jugador
33    1:
34      Si suma_dados<7 Entonces
35        Escribir "Ganaste"
36        capital := capital + apuesta
37      SiNo
38        Escribir "Perdiste"
39        capital := capital - apuesta
40      FinSi
41    2:
42      Si suma_dados=7 Entonces
43        Escribir "Ganaste"
44        capital := capital + 3*apuesta
45      SiNo
46        Escribir "Perdiste"
47        capital := capital - apuesta
48      FinSi
49    3:
50      Si suma_dados>7 Entonces
51        Escribir "Ganaste"
52        capital := capital + apuesta
53      SiNo
54        Escribir "Perdiste "
55        capital := capital - apuesta
56      FinSi
57  De Otro Modo:
58    Escribir "Selección incorrecta"

```

```
59   FinSegun  
60  
61   Escribir "Tu capital es de ",capital  
62   SiNo  
63     Escribir "No puedes apostar mas de lo que tienes..."  
64   FinSi  
65 FinProceso
```

A continuación, vamos a implementar el juego de piedra, papel o tijeras. Las reglas del juego para determinar quién gana según la selección hecha por cada jugador se presentan en el Cuadro 3.1. En caso de que ambos seleccionen la misma opción se declara un empate.

Ejemplo 3.18. Piedra, papel y tijeras (juego).

Para este juego se requieren dos jugadores. Cada uno de ellos debe seleccionar una de las opciones (piedra, papel o tijeras) de forma independiente. Las reglas del juego para determinar quién gana según la selección hecha por cada jugador se presentan en el Cuadro 3.1. En caso de que ambos seleccionen la misma opción se declara un empate.

Cuadro 3.1: Reglas del juego PPT.

Juegan		Regla	Ganador
Piedra	Papel	El papel cubre a la piedra	Papel
Piedra	Tijeras	La piedra rompe las tijeras	Piedra
Papel	Tijeras	Las tijeras rompen el papel	Tijeras

Para desarrollar un algoritmo que implemente este juego es necesario que cada jugador seleccione su opción sin que el otro se entere. Dado que esto no es posible, uno de los jugadores será la misma máquina que elegirá su opción sin considerar la opción del usuario y sin mostrársela para que éste no actúe en consecuencia. Así pues, al usuario se le preguntará por la opción que desea seleccionar mientras que la máquina deberá hacer la elección al azar.

Al usuario se le muestra una lista con las opciones: piedra, papel y tijeras; y un número asociado a cada opción: 1, 2 y 3, respectivamente. Se le pide que proporcione el número que corresponde a su elección (al listado de opciones se le llama **menú**). Para simular la elección de la máquina se utilizará la función `azar(3)` y cada uno de los posibles valores se hace corresponder con una opción: piedra, papel o tijeras. De esta manera, ya se tienen las elecciones de los dos jugadores y únicamente es necesario considerar las reglas del juego para elegir un ganador o declarar empate. El Algoritmo 3.27 contiene las instrucciones para implementar el juego.

Algoritmo 3.26: Piedra, papel y tijeras.

```
1 // Nombre: PPT.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Se implementa el juego de Piedra, Papel o Tijeras (PPT).
4
5 Proceso PPT
6   Definir jugador, maquina Como Entero
7
8   // selección de la máquina, regresa un valor entero 1, 2 o 3
9   maquina := azar(3)+1
10
11   // selección del usuario
12   Escribir "Este programa implementa el juego de Piedra Papel o Tijeras (PPT)"
13   Escribir "Teclea el numero que corresponde a tu selección"
14   Escribir "1) Piedra"
15   Escribir "2) Papel"
16   Escribir "3) Tijeras"
17   Leer jugador
18
19   Escribir "El jugador eligio ", jugador
20   Escribir "La maquina eligio ", maquina
21
22   Si maquina=jugador Entonces
23     Escribir "Empate"
24   SiNo
25     Segun jugador
26       1: // piedra
27         Segun maquina
28           2: // papel
29             escribir "Gana la máquina "
30           3: // tijeras
31             escribir "Gana el jugador"
32         FinSegun
33       2: // papel
34         Segun maquina
35           1: //piedra
36             escribir "Gana el jugador "
37           3: // tijeras
38             escribir "Gana la máquina "
39         FinSegun
40       3: // tijeras
41         Segun maquina
42           1: // piedra
43             escribir "Gana la máquina "
44           2: // papel
45             escribir "Gana el jugador"
46         FinSegun
47     De otro modo:
48       Escribir "Selección del jugador no válida"
49   FinSegun
50   FinSi
51 FinProceso
```

Ejemplo 3.19. Concatenar dos cadenas.**Algoritmo 3.27:** Concatenar cadenas.

```
1 // Nombre: Concatenar1.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Concatenación de dos cadenas dadas por el usuario.
4
5 Proceso ConcatenarNombreApellido
6   Definir nombre, apellido, nombre_completo como Caracter
7   Escribir "Como te llamas? " sin saltar
8   leer nombre
9   nombre := Mayusculas(nombre)
10  nombre_completo := nombre
11
12  nombre_completo := Concatenar(nombre_completo, ' ')
13
14  Escribir "Como te apellidas? " Sin Saltar
15  leer apellido
16  apellido := Mayusculas(apellido)
17
18  nombre_completo := Concatenar(nombre_completo, apellido)
19
20  Escribir "Tu nombre completo es ", nombre_completo
21 FinProceso
```

Con esto, termina el tema de la selección. En el siguiente capítulo se presentan las distintas formas de repetición de instrucciones dentro de un algoritmo.

Sugerencia de ejercicio.³

Sugerencia de ejercicio.⁴

NOTA⁵

³Convertir un número dado por el usuario [1,3999] a número romano

⁴Venta de boletos para el cine, dando como entrada el día y el mes, así como el sexo del cliente y su edad. Si es día del padre o de la madre, preguntarle si es padre o madre, según el sexo. Si se trata de una madre, y es el día de las madres, hacer un descuento del 50 %. Si es el día del padre, y el cliente es un padre, darle el 50 % de descuento. Si es el día del niño, hacerle un descuento del 100 % a todos los niños menores a 12 años.

⁵El EXAMEN1 incluye el material que se ha visto hasta el momento.

Capítulo 4 | Repetición

4.1. Ciclo Mientras Hacer

Este ciclo se vale de una condición (expresión lógica) para controlar la repetición del cuerpo del ciclo. Esta condición se encuentra antes del cuerpo del ciclo, lo cual significa que debe evaluarse la condición previamente para saber si el cuerpo del ciclo se ejecutará. El formato se muestra en la Figura 4.1. Si la condición es verdadera, se ejecutan las instrucciones que conforman el cuerpo del ciclo hasta que se encuentra la señal de finalización: **FinMientras**. En ese momento, el control regresa al inicio del ciclo, es decir, a la evaluación de la condición. Cuando la condición es falsa, no se ejecuta el cuerpo del ciclo y se rompe. Obsérvese que si la condición es falsa durante la primera evaluación, entonces no se ejecuta el cuerpo del ciclo ni una vez.

Una manera de utilizar este ciclo, es haciendo que la ejecución del bloque de instrucciones se repita n veces. Cuando se sabe de antemano el número de veces que se repetirá la ejecución del ciclo, se utiliza una variable que «cuente» el número de veces que se ha ejecutado, a este tipo de varia-

```
Mientras condición Hacer  
    // Bloque de instrucciones  
    // que se ejecutarán si  
    // condición es VERDADERO  
FinMientras
```

Figura 4.1: Estructura del ciclo Mientras-Hacer

bles se les llama **contadores**. Así pues, cuando el contador utilizado alcance un cierto valor, determinado de antemano, el ciclo se rompe. Por ejemplo, el siguiente segmento de código

```
Definir c Como Entero
c:=1
Mientras c<=5 Hacer
    Escribir "Hola, mundo."
    c:=c+1
FinMientras
```

utiliza la variable *c* como contador, el cual se inicializa en 1 y se incrementa de uno en uno con la operación $c := c + 1$ hasta alcanzar el valor 6, que hace que la condición controladora del ciclo sea FALSO y el ciclo se rompe. Así, se imprimirán los mensajes siguientes.

```
Hola, mundo.
Hola, mundo.
Hola, mundo.
Hola, mundo.
Hola, mundo.
```

Este ciclo se repite en cinco ocasiones. Cabe hacer notar que el control del ciclo está determinado por 3 operaciones sobre el contador: *inicialización*, *condición* e *incremento*, los cuales pueden manipularse para que se repitan el mismo número de veces aunque los valores generados para el contador sean diferentes. Así, el segmento anterior puede cambiarse por el siguiente

```
Definir c Como Entero
c:=15
Mientras c<=27 Hacer
    Escribir "Hola, mundo."
    c:=c+3
FinMientras
```

y obtener el mismo resultado (salida). Obsérvese que en este caso, el contador se inicializa en 15, se incrementa de 3 en 3 con la operación $c := c + 3$ y se rompe el ciclo cuando sobrepasa el valor 27. En general, un contador se incrementa con la operación

```
| contador := contador + constante
```

donde constante es un valor entero distinto de cero (negativo o positivo).

Así que es posible utilizar un contador que decremente su valor. En el siguiente segmento de código se ilustra este caso, obteniendo la salida deseada.

```
Definir c Como Entero  
c:=5  
Mientras c>0 Hacer  
    Escribir "Hola, mundo."  
    c:=c-1  
FinMientras
```

Si mandas imprimir el valor del contador, puedes generar un listado de números. El siguiente segmento de código genera la salida 1, 2, 3, 4, 5

```
Definir c Como Entero  
c:=1  
Mientras c<=5 Hacer  
    Escribir c, ", "  
    c:=c+1  
FinMientras
```

Un ciclo también puede usarse para cuidar el rango de valores válidos para una variable, por ejemplo, en el caso de que necesitemos leer el nombre del usuario y queremos asegurarnos de que no quede en blanco, se puede usar el siguiente segmento de código

```
Definir nombre como cadena
Escribir "Cómo te llamas?"
Leer nombre
Mientras Longitud(nombre)=0 Hacer
    Escribir "ERROR: El nombre del usuario no puede quedar en blanco"
    Escribir "Cómo te llamas? " Sin Saltar
    Leer nombre
FinMientras
// TO-DO: Implementar el programa cuando se cumple el criterio del nombre
```

Otro ejemplo sería, cuidar el valor de una variable numérica, como cuando leemos la edad de una persona y queremos asegurarnos de que sea un valor en un rango dado.

```
Definir edad como entero
Escribir "Cuántos años tienes? [0,100] " sin saltar
Leer edad
Mientras edad<0 o edad>100 Hacer
    Escribir "ERROR: Se espera un valor en [0,100]"
    Escribir "Cuántos años tienes? [0,100] " sin saltar
    Leer edad
FinMientras
// TO-DO: Implementar el programa cuando se cumple el criterio de la edad
```

Puede utilizar el ciclo Mientras-Hacer para asegurar que se elige una opción válida en un menú. Por ejemplo, en el siguiente segmento de código se muestra el menú de opciones al usuario y se le pide elegir una de ellas. Si el usuario elige una opción que no está en el listado, se le muestra un mensaje de error y se le da la oportunidad de elegir de nuevo, mediante el uso de un ciclo.

```
Definir opUsuario Como Entero
Escribir "Menú de opciones"
Escribir "1) Opción1"
Escribir "2) Opción2"
Escribir "3) Opción3"
Escribir "4) Opción4"
Escribir "Elige una opción [1,2,3 o 4] " Sin Saltar
Leer opUsuario

Mientras opUsuario<1 o opUsuario>4 Hacer
```

```

    Escribir "ERROR: se espera un valor en el rango [1,4]"
    Escribir "... presiona cualquier tecla para continuar"
    Esperar Tecla
    Limpiar Pantalla

    Escribir "Menú de opciones"
    Escribir "1) Opción1"
    Escribir "2) Opción2"
    Escribir "3) Opción3"
    Escribir "4) Opción4"
    Escribir "Elige una opción [1, 2, 3 o 4] " Sin Saltar
    Leer opUsuario
FinMientras
// TO-DO: Implementar lo que se va a hacer en cada una de las opciones del menu

```

Una variante del código anterior, que sirve para simplificar un poco el código, sería que nos aseguramos de siempre entrar al ciclo al inicializar la variable del usuario en un valor que haga que la condición que controla el ciclo sea verdadera (la primera vez que se evalúa). Una vez en el ciclo, se le muestra el menú y se comprueba si la elección del usuario es válida.

```

opUsuario:=10 // inicializado asi para asegurar que entre al ciclo
Esperar Tecla
Mientras opUsuario<1 o opUsuario>4 Hacer
    Escribir "Menú de opciones"
    Escribir "1) Opción1"
    Escribir "2) Opción2"
    Escribir "3) Opción3"
    Escribir "4) Opción4"
    Escribir "Elige una opción [1,2,3 o 4] " Sin Saltar
    Leer opUsuario
    si opUsuario<1 o opUsuario>4 Entonces
        Escribir "ERROR: se espera un valor en el rango [1,4]"
        Escribir "... presiona cualquier tecla para continuar"
        Esperar Tecla
        Limpiar Pantalla
    FinSi
FinMientras
// TO-DO: Implementar lo que se va a hacer en cada una de las opciones del menu.

```

El ciclo Mientras también puede usarse para hacer que un código se repita mientras el usuario así lo desee. El valor que se usa como controlador del ciclo se le llama **Valor centinela**. En el caso del siguiente segmento de código, el ciclo se repite mientras la respuesta del usuario sea 'S' o 'SI', por lo que esos valores son los centinelas del programa.

```

Definir resp Como Caracter
Escribir "Quieres jugar? (S/SI) " Sin Saltar
Leer resp
resp:=Mayusculas(resp)
Mientras resp='S' o resp='SI' Hacer

    // TO-DO: Aquí va la implementación del juego
    Escribir "... estoy jugando" // BORRAR esta línea

    Escribir "Quieres jugar otra vez? (S/SI) " Sin Saltar
    Leer resp
    resp:=Mayusculas(resp)
FinMientras

```

Es importante mencionar que los contadores pueden servir para contar la ocurrencia de algún suceso. Por ejemplo, considérese que se están leyendo valores y queremos contar cuántas veces aparece el número 0 en la captura. El siguiente segmento de código muestra cómo hacer esto: usamos un ciclo controlado por c1 donde, en cada iteración, se lee un valor y se revisa si es cero, en cuyo caso, se incrementa el valor de otro contador, c2. Al finalizar el ciclo, tendremos el c2 el número de 0's capturados por el usuario.

```

Definir c1,c2 Como Entero
Definir num Como Real
c1:=0
c2:=0
Mientras c1<=5 Hacer
    Escribir "proporciona un valor"
    Leer num
    Si num=0 Entonces
        c2:=c2+1
    FinSi
    c1:=c1+1
FinMientras
Escribir "Número de ceros capturados=",c2

```

Ejercicios

- Juego PPT usando un menú para las opciones a elegir del usuario (usando un ciclo Mientras para controlar que elija un valor válido: Piedra, Papel o Tijeras). El juego se repite mientras el usuario quiera seguir jugando.
- Juego Adivina el número mágico en [1,100]
 1. Mientras el usuario no adivine el número mágico, se le dará otra oportunidad. Cada vez que el jugador falle, se le dará una pista sobre si el número mágico es mayor o menor al número dado.
 2. Subir el nivel de complejidad del juego, al implementar un cierto número de oportunidades para que el jugador adivine.
 3. Inicialmente el rango de búsqueda del número mágico está en [1,100], pero conforme el usuario va descartando valores, se reduce el espacio de búsqueda, agregar al programa que cada vez que se le pide que adivine, se le muestre el rango de búsqueda actual.
- Juntar los juegos que se han implementado hasta el momento en un solo programa:
 1. Preguntar el usuario si quiere jugar.
 2. Si la respuesta es afirmativa, presentar al usuario un menú con las opciones para jugar PPT, El 7 y adivina el número mágico.
 3. Preguntar al usuario si quiere jugar de nuevo. El procedimiento se repite mientras el usuario quiera seguir jugando.

Otro tipo de variable, comúnmente utilizada en los ciclos, son los **acumuladores**, estos almacenan los valores acumulados de un conjunto de datos, comúnmente generados durante las iteraciones del ciclo. Por ejemplo, en el siguiente segmento de código

```
Definir c Como Entero
Definir suma,num Como Real
c:=0
suma:=0
Mientras c<5 Hacer
    Escribir Sin Saltar "proporciona un valor"
    Leer num
    suma := suma + num
    c := c + 1
FinMientras
Escribir "La suma es ",suma
```

se utiliza la variable *c* como contador, en el cuerpo del ciclo se solicita un valor al usuario, *num*, el cual una vez leído se acumula en una variable del tipo acumulador, *suma*, obsérvese que el valor almacenado en *suma* no se pierde sino que se ve incrementado por *num*. El valor inicial del acumulador generalmente es cero porque este valor no altera el resultado de la suma¹. Al romperse el ciclo, en *suma* se tiene la suma de todos los valores que proporcionó el usuario.

En general, la operación de acumulación tiene la siguiente forma:

```
| acumulador = acumulador + variable
```

donde *variable* es el valor acumulado; este valor puede ser negativo o positivo.

En el Ejemplo 4.1 se ilustra el funcionamiento de esta estructura de control utilizando un contador y un acumulador.

Ejemplo 4.1. Calcular el promedio de calificaciones de un grupo de 5 alumnos.

Supóngase que se tiene una calificación por alumno, para obtener el promedio es necesario sumaras y dividir las entre el número de alumnos

¹El valor inicial de un acumulador dependerá del uso que se le dé en el algoritmo.

(5 en este caso). Así, se requiere una variable para acumular las calificaciones, suma, y otra para contar el número de alumnos que se han considerado, c, la cual también controla el ciclo; además, se requiere una variable auxiliar con el propósito de leer la calificación del alumno en turno, calif. El Algoritmo 4.1 muestra una propuesta para resolver este problema. Es necesario inicializar la variable que contendrá la suma en 0, también la variable contadora debe tener el valor inicial 0 pues no se ha leído valor alguno. La condición de ejecución del ciclo queda determinada por la variable c ya que mientras tome un valor menor que 5, debe repetirse la ejecución del cuerpo del ciclo. Las instrucciones que forman parte del cuerpo del ciclo son: la solicitud y la lectura del valor de la calificación del alumno en cuestión, una vez obtenido éste se acumula en la variable suma y se incrementa en uno el valor del contador, c. Se repite el proceso hasta que c sea mayor o igual a 5. Una vez terminado el proceso de repetición, se utiliza la variable suma para calcular el promedio que se presenta para ser visualizado por el usuario.

Algoritmo 4.1: Promedio de grupo.

```
1 // Nombre: PromedioGrupo.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso PromedioGrupo
6   Definir prom,calif,suma ,c Como Real
7
8   suma:=0
9   c:=0
10  Mientras c<5 Hacer
11    Escribir Sin Saltar "Calificación ", c, " = "
12    Leer calif
13    suma := suma + calif
14    c := c + 1
15  FinMientras
16
17  prom := suma / c
18  Escribir "El promedio es ",prom
19 FinProceso
```

Ejercicio para el alumno²

²Modificar el Algoritmo 4.1 para que se verifique que cada calificación dada esté en un rango válido, por ejemplo de 0 a 100, antes de acumularla y contarla.

Ejemplo 4.2. Encontrar el mayor de 8 números.

Es posible utilizar 8 variables, una para cada número y después proceder a compararlos de una manera similar a la empleada en el Algoritmo 4.2. Es obvio, que usar este procedimiento resulta poco práctico sobre todo si aumenta el número de datos a comparar. En casos como éstos el uso de ciclos simplifica el proceso. Procederemos de la siguiente manera: El mayor de los números encontrados en cada iteración se almacena en una variable auxiliar declarada para ello, mayor. Se utiliza un contador, contador, que controle el número de iteraciones del ciclo, ocho en este caso, además se requiere una variable, num, para guardar el número leído en cada vuelta. El Algoritmo 4.2 ilustra las acciones realizadas.

Obsérvese que es necesario leer el primer número antes del ciclo para que éste sea considerado como el mayor hasta el momento. En el cuerpo del ciclo, cada vez que se lee un número, num, se compara con mayor, en caso de que éste sea mayor, se modifica el valor guardado en mayor sustituyéndolo con num. Finalmente, el valor mayor encontrado es presentado como salida del programa.

Algoritmo 4.2: Encontrar el mayor de 8 numeros dados.

```
1 // Nombre: MayorDeOcho.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso MayorDeOcho
6     Definir num, m, contador Como Entero
7
8     Escribir Sin Saltar "Numero 1 = "
9     Leer num
10    m := num
11    contador := 2
12    Mientras contador<=8 Hacer
13        Escribir Sin Saltar "Número ",contador," = "
14        Leer num
15        Si num > m Entonces
16            m := num
17        FinSi
18        contador := contador + 1
19    FinMientras
20    Escribir "El mayor es ",m
21 FinProceso
```

El ciclo Mientras puede usarse para presentar al usuario un menú de opciones. A continuación se presenta un ejemplo de este uso del ciclo.

Ejemplo 4.3. Presentar al usuario un menú de opciones para calcular el área de una figura geométrica. El menú contendrá un menú con 4 figuras geométricas a elegir.

Ejemplo 4.4. Sumar los números pares en el intervalo [1,100].

Se presentan dos versiones de un algoritmo que resuelva este problema. El Algoritmo 4.3 presenta una versión donde se requiere de una variable contadora, *i*, que tomará valores de 1 al 100. Además, se requiere una variable acumuladora, *suma*, en donde se acumulará el valor del contador cada vez que éste sea par. Finalmente, una vez terminado el ciclo se imprime el valor almacenado en *suma*.

Algoritmo 4.3: Suma de los pares en el intervalo mencionado.

```
1 // Nombre: SumarPares_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso SumarPares_a
6   Definir suma, i Como Entero
7   Escribir "Este programa calcula la suma de los números pares en [1,100]"
8   suma := 0
9   i := 1
10  Mientras i<=100 Hacer
11    Si i MOD 2 = 0 Entonces
12      suma := suma + i
13    FinSi
14    i := i + 1
15  FinMientras
16
17  Escribir "La suma es de los pares en [1,100] es ",suma
18 FinProceso
```

El Algoritmo 4.4 utiliza una variable para controlar el ciclo, *i*, la cual se inicializa en 2 y se incrementa de 2 en 2 hasta alcanzar el límite máximo permitido, 100. En este caso, *i* siempre toma un valor par, por lo que no es necesario hacer la verificación antes de acumularlo como en la versión anterior.

Algoritmo 4.4: Suma de los pares en el intervalo dado.

```

1 // Nombre: SumarPares_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso SumarPares_b
6   Definir suma, i Como Entero
7   Escribir "Este programa calcula la suma de los números pares en [1,100]"
8   suma := 0
9   i := 2
10  Mientras i<=100 Hacer
11    suma := suma + i
12    i := i + 2
13  FinMientras
14
15  Escribir "La suma es de los pares en [1,100] es ",suma
16 FinProceso

```

Ejemplo 4.5. Sumar los números pares y los impares (por separado) en el intervalo [1,100].

En la versión presentada por el Algoritmo 4.5, son necesarias dos variables, suma_pares y suma_impares, una para sumar los pares y la otra para los impares. Se utiliza un contador, c, para que tome todos los valores posibles en el rango dado. Al igual que en el Algoritmo 4.5, es necesario revisar la paridad del contador para saber si su valor se acumula a la suma de los pares o de los impares. Finalmente, cuando el ciclo se rompe, se imprimen los resultados obtenidos.

Algoritmo 4.5: Suma de los pares y los impares en el intervalo dado.

```

1 // Nombre: SumarParesImpares_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso SumarParesImpares_a
6   Definir suma_pares,suma_impares,c Como Entero
7
8   Escribir "Este programa calcula la suma de los pares e impares, por separado, "
9     Escribir "en el intervalo [1,100]"
10  suma_pares := 0
11  suma_impares := 0
12  c := 1
13  Mientras c<=100 Hacer
14    Si c MOD 2= 0 Entonces
15      suma_pares := suma_pares+c
16    Sino
17      suma_impares := suma_impares+c
18    FinSi
19    c := c + 1

```

```

20  FinMientras
21
22  Escribir "La suma de los pares es " ,suma_pares
23  Escribir "La suma de los impares es " ,suma_impares
24  FinProceso

```

En el Algoritmo 4.6 se presenta otra versión, en ésta se utilizan dos ciclos, el primero para calcular la suma de los pares y el segundo para la suma de los impares. Ambos ciclos, se controlan con el contador, *c*, ya que son independientes. En el primer ciclo, *c* se inicializa en dos y se incrementa de dos en dos para recorrer todos los números pares en el intervalo dado. En el segundo, *c* se inicializa en 1 y se incrementa de dos en dos para recorrer todos los impares en el intervalo dado. En cada caso se realiza la acumulación correspondiente.

Algoritmo 4.6: Suma de los pares y los impares en el intervalo dado.

```

1  // Nombre: SumarParesImpares_a.psc
2  // Autor: Irene Rodríguez y Edelmira Rodríguez
3  // Descripción:
4
5  Proceso SumarParesImparesB
6      Definir suma_pares,suma_impares,c Como Entero
7
8      Escribir "Este programa calcula la suma de los pares e impares, por separado, en el intervalo [1,100]"
9
10     suma_pares := 0
11     c := 2
12     Mientras c <= 100 Hacer
13         suma_pares := suma_pares + c
14         c := c + 2
15     FinMientras
16
17     suma_impares := 0
18     c := 1
19     Mientras c<=100 Hacer
20         suma_impares := suma_impares + c
21         c := c + 2
22     FinMientras
23
24     Escribir "La suma de los pares es " ,suma_pares
25     Escribir "La suma de los impares es " ,suma_impares
26 FinProceso

```

Ejemplo 4.6. Leer un conjunto de *n* números y contar los números pares que aparezcan.

Se requiere solicitar al usuario el número de datos que se leerán, n , y con la ayuda de un contador, c , en un ciclo se leerán los n datos proporcionados por el usuario, uno a la vez. Cada número leído se revisa para ver si es par en cuyo caso se incrementa un contador, numPares , que previamente fue inicializado en cero. Al finalizar el ciclo se imprime el valor del contador, numPares . En el Algoritmo 4.7 se muestra el código descrito.

Algoritmo 4.7: Contar el numero de pares leídos.

```

1 // Nombre: Contar_numero_Pares.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso Contar_numero_Pares
6   Definir num,c,n,numPares Como Entero
7
8   Escribir Sin Saltar "Cuántos números quieres capturar? "
9   Leer n
10  Si n<=0 Entonces
11    Escribir "Debe ser un valor positivo."
12    SiNo
13      c := 1
14      numPares := 0
15      Mientras c <= n Hacer
16        Escribir Sin Saltar "Numero="
17        Leer num
18        Si num MOD 2 = 0 Entonces
19          numPares := numPares + 1
20        FinSi
21        c := c + 1
22      FinMientras
23
24    Escribir "Se leyeron ",numPares," pares"
25  FinSi
26 FinProceso

```

Ejemplo 4.7. Cálculo del valor de π a partir de la serie infinita

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} \dots$$

Para comprender la situación, hagamos un análisis. Obsérvese que

- El numerador es constante e igual a 4 en cada término.
- El denominador tiene la forma $2k-1$ para $k=1,2,3, \dots$

- El signo de los términos (positivo o negativo) se alterna iniciando en positivo.

El número de términos utilizados para la aproximación, n , será proporcionado por el usuario. Si n es el número de términos que se desea utilizar y si k es el contador de términos que se han generado, entonces, k también nos servirá para obtener el denominador, pues cuando $k=1$, se obtiene el primer factor, $4/1$, si $k=2$ se obtiene el término $4/(2(2)-1)=4/3$, etc. Por tanto, cada uno de los términos tendrá la forma $4/(2k-1)$ donde k es un contador que toma valores desde 1 hasta n . Por otro lado, obsérvese que los términos con posición par se restan, mientras que los términos con posición impar se suman. Por ejemplo, si $k=1$, el signo es positivo, si $k=2$, el signo es negativo, etc.

Las variables que se requieren son un contador, k , el número de términos a considerar, n , una variable para el acumulador, mi_pi , y una variable para calcular el denominador, den . Todas son enteras excepto mi_pi , pues se están acumulando fracciones las cuales dan como resultado un valor real. En el Algoritmo 4.8 se muestra una versión que resuelve el problema.

Algoritmo 4.8: Cálculo del valor de PI a partir de la serie infinita dada.

```

1 // Nombre: CalcularPI_a.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso CalcularPI_a
6   Definir  $mi\_pi$  Como Real
7   Definir  $n, k, den$  Como Entero
8
9   Escribir "Este programa aproxima el valor de pi mediante una serie"
10  Escribir Sin Saltar "Cuántos términos de la serie deseas utilizar? "
11  Leer  $n$ 
12
13  Si  $n < 1$  Entonces
14    Escribir "El número de términos debe ser mayor o igual a 1"
15  SiNo
16     $mi\_pi := 4$ 
17     $k := 2$ 
18    Mientras  $k \leq n$  Hacer
19       $den := 2*k - 1$ 
20      Si  $k \bmod 2 = 0$  Entonces
21         $mi\_pi := mi\_pi - 4/den$ 
22      SiNo
23         $mi\_pi := mi\_pi + 4/den$ 
24      FinSi
25       $k := k + 1$ 

```

```

26      FinMientras
27
28      Escribir "Mi aproximación al valor de Pi = ",mi_pi
29  FinSi
30
31 FinProceso

```

En el Algoritmo 4.9 se parte de que $4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} \dots$ es equivalente a $4 + (-\frac{4}{3}) + \frac{4}{5} + (-\frac{4}{7}) + \frac{4}{9} + (-\frac{4}{11}) \dots$ para utilizar una variable auxiliar, signo, que tomará los valores de 1 ó -1 dependiendo del término generado. Se utilizará esta variable de la siguiente manera

$$\text{signo} * \frac{4}{2k-1}$$

con lo cual obtenemos el término a acumular. En cada iteración se modifica el valor de la variable signo multiplicándolo por 1, para alternar los signos. El término así obtenido se acumula en mi_pi.

Algoritmo 4.9: Cálculo del valor de PI a partir de la serie infinita dada.

```

1 // Nombre: CalcularPI_b.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso CalcularPI_b
6   Definir mi_pi Como Real
7   Definir n, k, den, signo Como Entero
8   Escribir "Este programa aproxima el valor de pi mediante una serie"
9   Escribir Sin Saltar "Cuántos términos de la serie deseas utilizar? "
10  Leer n
11
12  Si n < 1 Entonces
13    Escribir "El número de términos debe ser mayor o igual a 1"
14  SiNo
15    mi_pi := 4
16    signo := -1
17
18    k := 2
19    Mientras k <= n Hacer
20      den := 2*k - 1
21      mi_pi := mi_pi + signo*4/den
22      signo := signo * (-1)
23      k := k + 1
24    FinMientras
25
26    Escribir "Mi aproximación al valor de Pi = ",mi_pi
27  FinSi
28 FinProceso

```

Ejemplo 4.8. Multiplicar los números impares en el intervalo [1,10].

En la versión presentada en el Algoritmo 4.10, se requiere del uso de un contador, *c*, que tome los valores del 1 al 10. También se necesita una variable donde almacenar el resultado de la multiplicación realizada hasta el momento, *resultado*. La variable *resultado* se inicializa en uno porque servirá para calcular el valor parcial de la multiplicación, ya que es el neutro multiplicativo, y no debe tomar el valor cero que haría que el resultado fuera siempre cero. Una vez en el ciclo, se revisa si el valor del contador es impar en cuyo caso se multiplica por el valor obtenido hasta el momento para el resultado. Cuando se han revisado todos los valores en el rango especificado, se tendrá en resultado el valor deseado por lo que se presenta en la impresión.

Algoritmo 4.10: Multiplicar los impares leídos (1).

```
1 // Nombre: MultiplicarImparesA.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso MultiplicarImparesA
6   Definir resultado,c Como Entero
7   Escribir "Este programa calcula la multiplicación de los números impares en [1,10]"
8   resultado := 1
9   c := 1
10  Mientras c < 10 Hacer
11    Si c MOD 2 = 1 Entonces
12      resultado := resultado *c
13    FinSi
14    c := c + 1
15  FinMientras
16
17  Escribir "Resultado = ",resultado
18 FinProceso
```

En el Algoritmo 4.11 se inicializa el contador, *c*, en 1 y se incrementa de 2 en 2 hasta alcanzar el máximo valor en el rango dado. El valor del contador es el que se multiplica en cada iteración con el valor de resultado para obtener el valor resultado deseado, ya que éste, toma todos los valores impares de 1 al 10.

Algoritmo 4.11: Multiplicar los impares leídos (2).

```
1 // Nombre: MultiplicarImparesB.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
```

```

4
5 Proceso MultiplicarImparesB
6   Definir resultado,c Como Entero
7   Escribir "Este programa calcula la multiplicación de los números impares en [1,10]"
8   resultado := 1
9   c := 1
10  Mientras c < 10 Hacer
11    resultado := resultado *c
12    c := c + 2
13  FinMientras
14
15  Escribir "Resultado = ",resultado
16 FinProceso

```

Ejemplo 4.9. Encontrar el factorial de n.

En primer lugar, hay que recordar la fórmula para obtener el factorial de un número

$$n! = \begin{cases} 1, & \text{si } n = 0 \\ 1 \cdot 2 \cdot 3 \cdot 4 \cdots n, & \text{si } n \geq 1 \end{cases}$$

Obsérvese que el factorial para un número negativo no está definido y deberá considerarse este hecho en el algoritmo. En el Algoritmo 4.12 se muestra una propuesta de solución. Se utiliza una variable, contador, para controlar el ciclo y como auxiliar en el cálculo del factorial, f, ya que cada valor de contador será un término del producto acumulado en cada iteración del ciclo.

Algoritmo 4.12: Factorial de n.

```

1 // Nombre: FactorialN.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso FactorialN
6   Definir f,n,contador Como Entero
7
8   Escribir "Este programa calcula el factorial de un número entero dado"
9   Escribir Sin Saltar "n ="
10  Leer n
11  Si n<0 Entonces
12    Escribir "El factorial no está definido para valores negativos"
13  SiNo
14    f := 1
15

```

```
16     contador := 2
17     Mientras contador<=n Hacer
18         f := f * contador
19         contador := contador + 1
20     FinMientras
21
22     Escribir "El factorial de ",n, " es " ,f
23 FinSi
24
25 FinProceso
```

Un error típico en el diseño de una sentencia Mientras-Hacer es que el cuerpo del ciclo no contenga una sentencia que modifique la situación de la condición que lo controla. El siguiente segmento de código ejemplifica esta situación:

```
contador := 1
Mientras contador <3 Hacer
    Escribir "contador = ",contador
FinMientras
contador := contador + 1
```

En este caso, la variable contador nunca cambia su valor en el cuerpo del ciclo por lo que la condición siempre será verdadera. De este modo, se obtiene un ciclo que se repite infinitamente. Debe tenerse cuidado de que el cuerpo del ciclo siempre contenga las instrucciones necesarias para que la condición que controla el ciclo sea, en algún momento, falsa y el ciclo se rompa. Otro caso es que el valor del contador cambie pero que esté alternando su valor mediante una operación de inicialización dentro del cuerpo del ciclo como en el siguiente bloque de instrucciones

```
Mientras c<10 Hacer
    c := 1
    Escribir "Contador=", contador
    c := c + 1
fin mientras
```

Suponiendo que la condición es verdadera en la primera evaluación el valor del contador, c, se inicializa en 1. Dentro del cuerpo del ciclo, se imprime y se incrementa para tomar el valor 2, después se evalúa la condición del ciclo que es verdadera de nuevo y se inicializa en uno otra vez con

lo cual el valor del contador estará alternando entre 1 y 2 y la condición nunca será falsa, es decir, será un ciclo infinito.

4.1.1. Variantes del uso del ciclo

Puesto que el cuerpo del ciclo Mientras-Hacer utiliza una condición para controlar la repetición, es posible utilizar una variable que tome valores VERDADERO o FALSO como controlador del flujo de acciones. Este tipo de variable se conoce como lógica o booleana y se declara de la siguiente manera:

```
Definir identificador Como Logico
```

donde, `identificador` es el nombre de la variable lógica.

La forma de controlar el ciclo Mientras-Hacer mediante una variable lógica (comúnmente llamada indicadora o bandera) es asignar un valor inicial VERDADERO a dicha variable y hacer que la condición que controle el ciclo sea precisamente que la variable tome el valor VERDADERO y en algún momento de la ejecución del cuerpo del ciclo modificar el valor de la bandera a FALSO con lo cual se romperá el ciclo o viceversa (se inicializa como FALSO, la condición sería que la bandera tomara el valor FALSO y en algún momento se cambia a VERDADERO para que el ciclo se rompa).

En el siguiente segmento de código ilustramos la situación antes mencionada. Se leerán datos de tipo carácter mientras el carácter leído sea distinto del '0' (recuérdese que al trabajar con caracteres constantes es necesario utilizar los apóstrofes). La variable `ContinuarLectura` es de tipo lógico y se inicializa en VERDADERO, se usa como condición del ciclo que `ContinuarLectura = VERDADERO` con lo cual aseguramos que se ejecutará el cuerpo del ciclo al menos en una ocasión; dentro del cuerpo del ciclo se lee un carácter y si se trata del '0' se cambia `ContinuarLectura` a FALSO con lo cual se rompe el ciclo.

```
Definir dato Como Carácter  
Definir ContinuarLectura Como Logico
```

```
ContinuarLectura := VERDADERO
Mientras ContinuarLectura Hacer
    Escribir "Teclea un caracter"
    Leer dato
    Si dato = '0' Entonces
        ContinuarLectura := FALSO
    FinSi
FinMientras
```

Ejemplo 4.10. Encontrar los divisores de un número entero dado por el usuario.

```
Escribir "Divisores de ",n
contador := 1
Mientras contador <= n Hacer
    Si n MOD contador = 0 Entonces
        Escribir contador
    FinSi
    contador := contador + 1
FinMientras
```

Ejemplo 4.11. Indicar si un número entero positivo dado es primo.

Se pretende revisar a todos los posibles divisores de un número dado n , se utilizará una bandera, *primo*, inicializada en VERDADERO que indicará con esto que el número n es efectivamente primo. En el proceso de revisión, realizado en un ciclo, se toman todos los posibles divisores de n , que son 2, 3, ..., $n-1$, y en el momento en que se encuentre uno que sea divisor de n , se modifica el valor de la bandera a FALSO, luego ya fuera del ciclo se imprime un mensaje dependiendo del valor de la bandera. Si la bandera tiene el valor VERDADERO se imprime que el número es primo, de lo contrario se imprime que no es primo.

Algoritmo 4.13: Identificar si un número dado es primo.

```
1 // Nombre: IdentificarPrimo.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso IdentificarPrimo
6     Definir primo Como Logico
7     Definir n, contador Como Entero
```

```

8
9  Escribir "Determinar si un número dado es primo"
10 Escribir Sin Saltar "Número ="
11 Leer n
12
13 Si n >= 1 Entonces
14     primo := VERDADERO
15     contador := 2
16     Mientras contador <= n - 1 y primo = VERDADERO Hacer
17         Si n MOD contador = 0 Entonces
18             primo := FALSO
19         FinSi
20         contador := contador + 1
21     FinMientras
22
23 Si primo = VERDADERO Entonces
24     Escribir "El número ",n," es primo"
25 SiNo
26     Escribir "El número ",n," no es primo"
27 FinSi
28 SiNo
29     Escribir "El valor dado debe ser positivo"
30 FinSi
31
32 FinProceso

```

Comúnmente, no se conoce de antemano el número de veces que se va a ejecutar el cuerpo de un ciclo. Esto puede suceder cuando el usuario está capturando valores en cada iteración y no sabe cuántos son exactamente o bien cuando los datos siguientes depende de los resultados obtenidos en la iteración previa. Para estos casos, es posible aleccionar al usuario para que proporcione un valor que indicará que la lectura de los datos ha terminado, es decir, existirá un valor que actuará como centinela para terminar el ciclo. En el Ejemplo 4.14 se presenta esta situación.

Ejemplo 4.12. Encontrar las parejas de números amigos menores a 2,000.

DEFINICIÓN: Los números amigos o amigables son dos números naturales a y b cuya suma de los divisores de uno de ellos (sin incluir al número) es igual al otro número, y la suma de los divisores de este otro (no incluyéndolo tampoco) es igual al primer número.

Ejemplos de números amigos:

- 220 y 284

- 1184 y 1210
- 2620 y 2924

Ejemplo 4.13. Encontrar los números perfectos menores a 1,000.

DEFINICIÓN: Un número perfecto es un número entero positivo que es igual a la suma de sus divisores propios positivos. Dicho de otra forma, un número perfecto es aquel que es amigo de sí mismo.

Ejemplos de números perfectos:

- $6 = 1 + 2 + 3$
- $28 = 1 + 2 + 4 + 7 + 14$
- $496 = 1 + 2 + 4 + 8 + 16 + 31 + 62 + 124 + 248$
- $8128 = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 127 + 254 + 508 + 1016 + 2032 + 4064$

Ejemplo 4.14. Promedio de calificaciones usando un valor centinela.

Dentro de un ciclo se leerá una calificación a la vez la cual se acumula en suma y se incrementa el valor del contador. Cuando se solicita la calificación al usuario se le instruye para que, en caso de que no haya otra calificación, introduzca el -1 como centinela. La condición de continuación del ciclo es que la calificación proporcionada sea distinta del valor centinela. Por lo tanto, cuando el usuario proporcione el valor -1 se romperá el ciclo y se calculará el promedio si se ejecutó el cuerpo del ciclo al menos en una ocasión. En este caso, -1 es un buen valor centinela puesto que se considera que no hay calificaciones negativas. Pudo utilizarse en su lugar cualquier valor que esté fuera del rango considerado como válido. Es una buena práctica de programación indicar el valor centinela en cada mensaje de solicitud de datos adecuado.

Algoritmo 4.14: Uso de un valor centinela (promedio de calificaciones).

```
1 // Nombre: UsandoCentinela1.psc
```

```

2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Este programa calcula el promedio de un conjunto de calificaciones.
4 //      No se conoce de antemano cuantas calificaciones se van a leer, sino que se revisa
5 //      si el valor capturado es el valor -1, con lo que se indica que la lista a leer
6 //      ha finalizado.
7
8 Proceso UsandoCentinela1
9     Definir calif, suma Como Real
10    Definir contador Como Entero
11
12    contador := 0
13    suma := 0
14
15    Escribir "Este programa calcula el promedio de una serie de calificaciones dadas "
16    Escribir Sin Saltar "Escribe una calificación [para terminar, teclea un -1]"
17    Leer calif
18
19    Mientras calif != -1 Hacer
20        Si calif >= 0 y calif <= 100 Entonces
21            contador := contador + 1
22            suma := suma + calif
23        FinSi
24
25        Escribir "Escribe la siguiente calificación [para terminar, teclea un -1]"
26        Leer calif
27    FinMientras
28
29    Si contador > 0 Entonces
30        Escribir "El promedio es ", suma/contador
31    FinSi
32 FinProceso

```

También puede utilizarse un carácter como valor centinela. En el Ejemplo 4.15 se utiliza de esta manera el ciclo Mientras-Hacer.

Ejemplo 4.15. Promedio de calificaciones usando un carácter como centinela.

Dentro de un ciclo se lee una calificación a la vez, se acumula en suma y se cuenta con contador. En esta ocasión, el ciclo está controlado por la condición `resp='s'`, previamente se ha inicializado la variable `resp` con el carácter proporcionado por el usuario como respuesta a la pregunta *¿hay alguna calificación?* [s/n]. Obsérvese que se han indicado como parte del mensaje los valores esperados para la respuesta los cuales pueden ser 's' o 'n'. El valor de la variable `resp` se actualiza dentro del cuerpo del ciclo al preguntarle al usuario *¿hay otra calificación?* [s/n] con lo cual damos pie a que en algún momento se rompa el ciclo al proporcionar el usuario al-

gún valor distinto de 's'. Obsérvese también que se acepta el caracter 's' y el caracter 'S' como señal de fin del ciclo.

Algoritmo 4.15: Uso de un carácter como valor centinela.

```

1 // Nombre: UsandoCentinela2.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción: Este programa calcula el promedio de un conjunto de calificaciones.
4 // No se conoce de antemano cuantas calificaciones se van a leer, en cada iteracion del
5 // ciclo se pregunta si hay otra calificacion a leer, el ciclo se repite si la respuesta
6 // es afirmativa ('s' o 'S'), en caso contrario (cuando se responde con una letra
7 // distinta de 's' o 'S'), el ciclo se rompe.
8
9 Proceso UsandoCentinela2
10   Definir calif,suma Como Real
11   Definir resp Como Caracter
12   Definir contador Como Entero
13
14   contador := 0
15   suma := 0
16   Escribir Sin Saltar "Este programa calcula el promedio de un conjunto de calificaciones"
17   Escribir Sin Saltar "Hay alguna calificación? [s/n]="
18   Leer resp
19
20   Mientras resp='s' o resp='S' Hacer
21     Escribir Sin Saltar "Calificación ="
22     Leer calif
23     Si calif>=0 y calif<=100 Entonces
24       contador := contador + 1
25       suma := suma + calif
26     FinSi
27     Escribir Sin Saltar "Hay otra calificación? [s/n] ="
28     Leer resp
29   FinMientras
30
31   Si contador>1 Entonces
32     Escribir "El promedio es ", suma/contador
33   FinSi
34
35 FinProceso

```

Incluso se puede considerar una variante del algoritmo anterior, utilizando en lugar de la condición del ciclo que se acepte la letra mayúsculas 's' y 'S', una vez leída la respuesta se convierte a mayúsculas usando la función MAYUSCULAS() ya incluida en el PSeInt. Considerar el siguiente segmento de código donde se utiliza esta función. De esta manera , solo debe compararse el valor leído con el caracter 'S'.

```

Escribir Sin Saltar "Hay alguna calificacion? [s/n]="
Leer resp

```

```

Mientras MAYUSCULAS(resp)='S' Hacer
    Escribir Sin Saltar "Hay otra calificacion? [s/n] ="
    Leer resp
FinMientras

```

Aunque el ciclo Mientras-Hacer nos ha sido útil cuando se conoce de antemano el número de veces que se ejecutará, existe otro ciclo más adecuado para esos casos, el ciclo para-hacer que se presenta en la sección siguiente.

4.2. Ciclo Para-Hacer

Este ciclo se utiliza cuando se sabe de antemano el número de veces que se va a repetir el ciclo. Se utiliza una variable, contador, para controlar el ciclo mediante las operaciones de inicialización, condición e incremento. En la inicialización se asigna el *ValorInicial* al contador, el cual se incrementa hasta su *ValorFinal*, en cada iteración se incrementa en una cantidad especificada en *Incremento*. En la Figura 4.2 se ilustra esta estructura de control.

```

Para Contador Desde ValorInicial Hasta ValorFinal Con Paso Incremento Hacer
    Bloque de instrucciones a realizar en cada iteración
FinPara

```

Figura 4.2: Estructura del ciclo **Mientras-Hacer**

Aunque el formato de este ciclo permite algunas variantes, por ejemplo, si el incremento de la variable contador es de uno, puede omitirse, quedando el ciclo como sigue

```

Definir Contador Como Entero
Para Contador Desde ValorInicial Hasta ValorFinal Hacer
    //Bloque de instrucciones a realizar en cada iteracion
FinPara

```

Otra variante permitida está relacionada con la forma en que se inicializa la variable Contador, permitiendo usar el operador de asignación, como se ilustra a continuación

```

Definir Contador Como Entero
Para Contador<-ValorInicial Hasta ValorFinal Hacer
    //Bloque de instrucciones a realizar en cada iteracion
FinPara

```

Es importante notar que el valor de Incremento puede ser negativo, sirviendo en este caso para hacer que la variable tome valores que decremen-ten desde el ValorInicial hasta el ValorFinal.

En el Ejemplo 4.9 se analizó el problema de calcular el factorial de un número entero, n . En el Algoritmo 4.16 se muestra una versión utilizando el ciclo Para-Hacer

Algoritmo 4.16: Factorial de n .

```

1 // Nombre: FactorialN_v2.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso FactorialN_v2
6     Definir f,n,c Como Entero
7
8     Escribir "Este programa calcula el factorial de un número entero dado"
9     Escribir Sin Saltar "n="
10    Leer n
11
12    Si n<0 Entonces
13        Escribir "El factorial no está definido para valores negativos"
14    SiNo
15        f := 1
16
17        Para c Desde 2 Hasta n Con Paso 1 Hacer
18            f := f * c
19        FinPara
20
21        Escribir "El factorial de ",n, " es " ,f
22    FinSi
23 FinProceso

```

Si se compara el ciclo Mientras -Hacer con el ciclo Para-Hasta-Hacer, se observa que todos los elementos están ahí, si se conoce el valor inicial y el valor final del contador, son estructuras equivalentes.

Ejemplo 4.16. Simular el lanzamiento de una moneda.

Lanzar la moneda 100 veces y contar el número de veces que cayó cara y el número de veces que cayó cruz. La simulación del lanzamiento de una moneda se realiza utilizando la función **aleatorio**(0,1), en caso de que la función regrese un 1, se considera que se obtuvo una cara y se cuenta su ocurrencia utilizando el contador, NumCaras; en caso contrario, se considera que cayó una cruz y se cuenta por separado, utilizando el contador NumCruces. En el Algoritmo 4.17 se representa esta situación. Para simular el número de lanzamientos de la moneda se utiliza un ciclo **para-hacer** controlado por un contador, c, el cual toma valores desde 1 hasta 100 con incrementos de 1 en 1; así, el ciclo se repetirá 100 veces.

Algoritmo 4.17: Lanzamiento de una moneda 100 veces.

```

1 // Nombre: LanzamientoMoneda.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso LanzamientoMoneda
6   Definir NumCaras,NumCruces,c,moneda Como Entero
7
8   Escribir "Este programa simula el lanzamiento de una moneda 100 veces"
9
10  NumCaras:=0
11  NumCruces:=0
12
13  Para c<-1 Hasta 100 Hacer
14    moneda := Aleatorio(0,1)
15    Si moneda=1 Entonces
16      NumCaras := NumCaras + 1
17    SiNo
18      NumCruces := NumCruces + 1
19    FinSi
20  FinPara
21
22  Escribir "El número de caras obtenidas es ",NumCaras
23  Escribir "El número de cruces obtenidas es ",NumCruces
24
25 FinProceso

```

Ejemplo 4.17. Generar n términos de la Sucesión de Fibonacci.

La Sucesión de Fibonacci inicia con los valores 0 y 1 y obtiene el siguiente valor de la sucesión sumando los últimos dos valores obtenidos, así, por ejemplo, los primeros 8 términos de la serie son 0, 1, 1, 2, 3, 5, 8, 13.

El algoritmo necesario para resolver este problema, requiere una variable, n, que contenga el número de términos deseados. También se requieren dos variables, penultimo y ultimo, que almacenen los últimos dos valores obtenidos de la sucesión. Además, se utiliza una variable auxiliar, siguiente, para almacenar el siguiente valor obtenido. El valor siguiente se obtiene sumando los valores de penultimo y ultimo. Inicialmente, penultimo=0 y ultimo=1, así que restan por obtener n-2 términos. En el Algoritmo 4.18 se presentan las instrucciones requeridas para resolver este problema.

Algoritmo 4.18: Generar la sucesión de Fibonacci

```
1 // Nombre: Fibonacci.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso Fibonacci
6   Definir siguiente,ultimo,penultimo,c,n Como Entero
7
8   Escribir "Este programa genera n términos de la Sucesión de Fibonacci"
9   Escribir Sin Saltar "n="
10  Leer n
11
12  Si n>=1 Entonces
13    penultimo := 0
14    ultimo := 1
15    Escribir penultimo
16    Escribir ultimo
17
18    Para c<-3 Hasta n Hacer
19      siguiente := ultimo+penultimo
20      Escribir siguiente
21
22      penultimo := ultimo
23      ultimo := siguiente
24    FinPara
25  SiNo
26    Escribir "Se requiere un valor positivo"
27  FinSi
28 FinProceso
```

Ejemplo 4.18. Determinar si un número entero positivo dado es primo.

Recuérdese que un número es primo cuando, solamente, es divisible entre 1 y entre sí mismo. Así que, si encontramos que existe al menos un divisor adicional, sabremos que el número considerado no es primo. En el Algoritmo 4.19, se lee un número, n y se parte de la suposición de que n es primo, para lo cual se utiliza una bandera, `primo`, que se inicializa en VERDADERO. Se consideran como posibles divisores todos los valores enteros entre 2 y $n-1$, por lo cual se utiliza un ciclo **ParaHacer**, para generar estos valores con un contador. En caso de que se encuentre un divisor entre éstos, se cambia el valor de la bandera a FALSO y se hace que el contador tome el valor n para obligar a que se rompa el ciclo pues con ese valor la condición controladora será falsa en la siguiente evaluación. El valor de la bandera al finalizar el ciclo determina el mensaje enviado al usuario.

Algoritmo 4.19: Determinar si un número entero positivo dado es primo.

```

1 // Nombre: Primo.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso IdentificarPrimo
6   Definir primo Como Logico
7   Definir n, contador Como Entero
8
9   Escribir "Determinar si un número dado es primo"
10  primo := VERDADERO
11  Escribir Sin Saltar "Número ="
12  Leer n
13
14  Si n >= 1 Entonces
15
16    contador := 2
17    Mientras contador <= n - 1 y primo = VERDADERO Hacer
18      Si n MOD contador = 0 Entonces
19        primo := FALSO
20      FinSi
21      contador := contador + 1
22    FinMientras
23
24    Si primo = VERDADERO Entonces
25      Escribir "El número ",n," es primo"
26    SiNo
27      Escribir "El número ",n," no es primo"
28    FinSi
29  SiNo
30    Escribir "El valor dado debe ser positivo"
31  FinSi
32

```

En el siguiente ejemplo, se calcula la integral de una función $f(x)$ en el intervalo $[a, b]$ con el método de aproximaciones utilizando rectángulos inferiores.

Ejemplo 4.19. Calcular una integral numérica inferior en el intervalo $[a, b]$.

Utilizar la función $f(x) = x^2 + 1$ en el intervalo $[a, b]$ dado. Para resolver el problema debemos recordar como se calcula una integral numérica inferior. En la Fig. IV-5 se visualiza la partición del intervalo $[a, b]$ en n subintervalos, cada uno de estos tiene un ancho de $dx = (b - a) / n$. Para obtener la aproximación al área bajo la curva se emplean rectángulos formados por las particiones hechas y se suma el área de cada uno de ellos. El área de cada rectángulo es igual a la altura del rectángulo por la longitud de la base. La altura es igual al valor de la función en x , donde $x = a + i * dx$ con $i = 0, 1, \dots, n - 1$ y la base es dx . Obsérvese que el valor de la aproximación es mejor conforme hacemos que n tome un valor mayor.

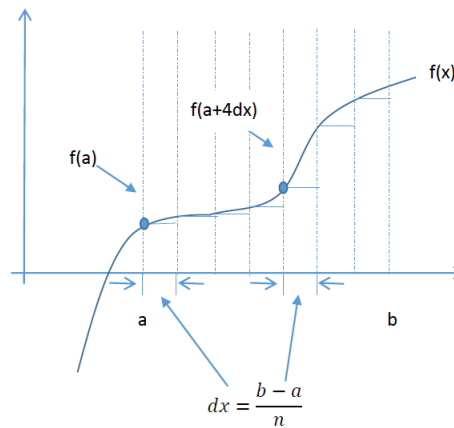


Figura 4.3: Aproximación al área bajo la curva de $f(x)$.

A este método para aproximar la integral bajo una curva se le conoce como Sumas de Riemman, y en este sitio de [Khan Academy](#) se explica mejor.

En los Algoritmos 4.20 y 4.21 se leen los valores extremos del intervalo, a y b , además del número de particiones requeridas. Se espera que $a \leq b$ y que $n > 0$, si esto sucede entonces se procede a calcular la aproximación de la integral. Para esto, se calcula la longitud de la partición, dx , como la longitud del intervalo $[a, b]$ entre el número de particiones. Utilizaremos una variable, x , que tome como valor todos los puntos iniciales de los subintervalos, además, x será el valor en que evaluaremos la función para encontrar la altura de cada rectángulo. Inicialmente le asignamos a x el valor a y en cada iteración se incrementa en dx , para que tome el valor del punto inicial del siguiente subintervalo. Para simular la suma de las áreas de cada rectángulo se utiliza un acumulador, *integral*, inicializado en cero, y cuyo valor se incrementa en el área de cada rectángulo calculada como el producto de la función evaluada en x por dx .

En el Algoritmo 4.20 se utiliza un ciclo controlado por un contador, i , con valores de 1 a n , para hacer que el ciclo se repita n veces.

Algoritmo 4.20: Calcular una integral numérica inferior.

```

1 // Nombre: IntegralInferior_v1.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso IntegralInferior_v1
6   Definir n,i Como Entero
7   Definir dx,integral,x,a,b Como Real
8
9   Escribir "Calcula el valor de la integral bajo la curva"
10
11  Escribir Sin Saltar "Límite inferior del intervalo = "
12  Leer a
13
14  Escribir Sin Saltar "Límite superior del intervalo = "
15  Leer b
16
17  Escribir Sin Saltar "Número de particiones = "
18  Leer n
19
20  Si n > 0 y a <= b Entonces
21    dx := (b-a)/n
22    x := a
23    integral := 0
24    Para i <- 1 Hasta n Hacer
25      integral := integral + ( x * x + 1 ) * dx
26      x := x + dx
27    FinPara
28    Escribir "La integral bajo la curva es ",integral
29  SiNo
30    Escribir "ERROR: Los limites del intervalo o el número de particiones es incorrecto"

```

```

31 FinSi
32 FinProceso

```

valores reales. En el Algoritmo 4.21 se utilizan como controlador del ciclo a la variable x . El punto inicial del último subintervalo se encuentra en $b-dx$, por lo cual el ciclo debe terminar cuando x tome el valor b . Obsérvese que las variables x y dx son de tipo Real, por lo que los incrementos son valores reales (no necesariamente valores enteros).

Algoritmo 4.21: Calcular una integral numérica inferior.

```

1 // Nombre: IntegralInferior_v2.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso IntegralInferior_v2
6   Definir n,i Como Entero
7   Definir dx,integral,x,a,b Como Real
8
9   Escribir "Determinar si un número dado es primo"
10
11   Escribir Sin Saltar "Límite inferior del intervalo = "
12   Leer a
13
14   Escribir Sin Saltar "Límite superior del intervalo = "
15   Leer b
16
17   Escribir Sin Saltar "Número de particiones = "
18   Leer n
19
20   Si n > 0 y a <= b Entonces
21     dx := (b-a)/n
22     integral := 0
23     Para x <- a Hasta b Con Paso dx Hacer
24       integral := integral + ( x * x + 1 ) * dx
25     FinPara
26     Escribir "La integral bajo la curva es ",integral
27   SiNo
28     Escribir "ERROR: Los límites del intervalo o el número de particiones es incorrecto"
29   FinSi
30 FinProceso

```

4.2.1. Figuras geométricas

Ejemplo 4.20. Leer dos valores enteros positivos, que representen el ancho y lo alto de la figura que se va a formar. La salida es un cuadrado dibujado con asteriscos con las dimensiones dadas por el usuario.

Algoritmo 4.22: Dibuja un rectángulo.

```

1 // Nombre: DibujandoRectangulo.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso Rectangulo
6     Definir m,n,i,j como Entero
7     Definir s como Caracter
8
9     Escribir "Este programa dibuja un rectángulo con el caracter elegido "
10    Escribir "y las dimensiones dadas"
11    Escribir "Alto=" Sin Saltar
12    Leer m
13    Escribir "Ancho=" sin saltar
14    Leer n
15    Escribir "Caracter de tu elección para hacer el dibujo="
16    Leer s
17
18    Para i<-1 hasta m Hacer
19        para j<-1 hasta n hacer
20            Escribir s sin saltar
21        FinPara
22        Escribir ""
23    FinPara
24 FinProceso

```

```

PSeInt - Ejecutando proceso CUADRADO
*** Ejecución Iniciada. ***
Este programa dibuja un rectángulo con el caracter elegido y las dimensiones dadas
Alto=> 5
Ancho=> 9
Caracter de tu eleccion para hacer el dibujo=
> *
*****
*****
*****
*****
*****
*** Ejecución Finalizada. ***
☐ No cerrar esta ventana ☐ Siempre visible Reiniciar

```

Figura 4.4: Rectángulo

Ejemplo 4.21. Leer dos valores enteros positivos, que representen el ancho y lo alto de la figura que se va a formar. La salida es un triángulo dibujado con asteriscos con las dimensiones dadas por el usuario.

Algoritmo 4.23: Dibuja un Triángulo.

```

1 // Nombre: DibujandoTriangulo1.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso Triangulo
6     Definir m,n,i,j como Entero
7     Definir s como Caracter
8
9     Escribir "Este programa dibuja un triángulo con el caracter elegido y la altura dada"
10    Escribir "Altura del triangulo=" Sin Saltar
11    Leer m
12
13    Escribir "Caracter de tu elección para hacer el dibujo="
14    Leer s
15
16    Limpiar pantalla
17    Para i<-1 hasta m Hacer
18        para j<-1 hasta i hacer
19            Escribir s sin saltar
20        FinPara
21        Escribir ""
22    FinPara
23
24 FinProceso

```

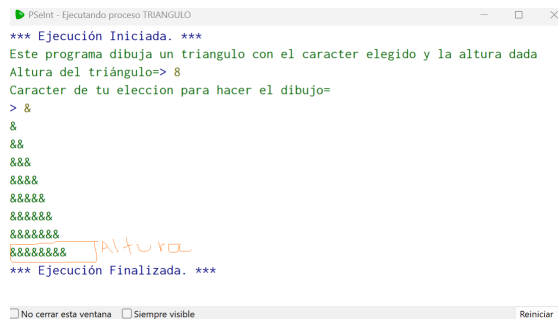


Figura 4.5: Triangulo1

Ejemplo 4.22. Leer dos valores enteros positivos, que representen el ancho y lo alto de la figura que se va a formar. La salida es un triángulo dibujado con asteriscos con las dimensiones dadas por el usuario.

Algoritmo 4.24: Dibuja un Triangulo (2).

```

1 // Nombre: DibujandoTriangulo2.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:

```

```

4
5 Proceso Triangulo2
6   Definir m,n,i,j como Entero
7   Definir s como Caracter
8
9   Escribir "Este programa dibuja un triángulo con el caracter elegido y la altura dada"
10  Escribir "Altura del triangulo=" Sin Saltar
11  Leer m
12
13  Escribir "Caracter de tu elección para hacer el dibujo="
14  Leer s
15  Escribir ""
16  //Limpiar pantalla
17  Para i<-1 hasta m Hacer
18      para j<-1 hasta i hacer
19          Escribir s sin saltar
20      FinPara
21      Escribir ""
22  FinPara
23  Para i<-m-1 hasta 1 Hacer
24      para j<-1 hasta i hacer
25          Escribir s sin saltar
26      FinPara
27      Escribir ""
28  FinPara
29 FinProceso

```

```

PSeInt - Ejecutando proceso TRIANGULO2
*** Ejecución Iniciada. ***
Este programa dibuja un triangulo con el caracter elegido y la altura dada
Altura del triángulo=> 7
Caracter de tu eleccion para hacer el dibujo=
> +

+
++
+++
++++
+++++
++++++
+++++++
+++++++ ← Altura
+++++++
+++++++
++++
+++
++
+

*** Ejecución Finalizada. ***
☐ No cerrar esta ventana ☐ Siempre visible Reiniciar

```

Figura 4.6: Triangulo 2

Ejemplo 4.23. Leer dos valores enteros positivos, que representen el ancho y lo alto de la figura que se va a formar. La salida es un triángulo dibujado con asteriscos con las dimensiones dadas por el usuario.

Algoritmo 4.25: Dibuja un Triangulo (3).

```

1 // Nombre: TrianguloInvertido.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso TrianguloInvertido1
6   Definir m,n,i,j como Entero
7   Definir s como Caracter
8
9   Escribir "Este programa dibuja un triángulo con el caracter elegido y la altura dada"
10  Escribir "Altura del triangulo=" Sin Saltar
11  Leer m
12
13  Escribir "Caracter de tu elección para hacer el dibujo="
14  Leer s
15  Escribir ""
16  //Limpiar pantalla
17  Para i<-1 hasta m Hacer
18    para j<-1 hasta m-i con paso 1 Hacer
19      Escribir " " sin saltar
20    FinPara
21    para j<-1 hasta i con paso 1 hacer
22      Escribir s sin saltar
23    FinPara
24    Escribir ""
25  FinPara
26  Escribir ""
27 FinProceso

```

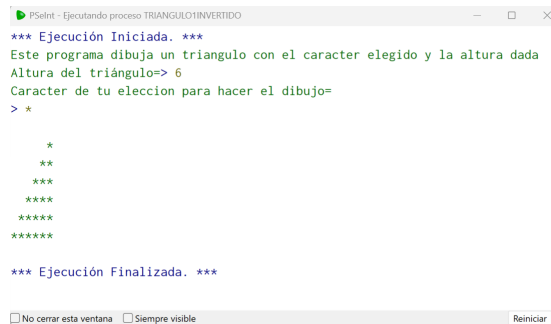


Figura 4.7: Triangulo 3

Como ejercicios, se sugiere programar las siguientes figuras

En la siguiente sección, se presenta la tercera, y última, estructura de repetición: el ciclo **RepetirHasta**. Este es una versión modificada del ciclo **MientrasHacer**.

4.3. Repetir-Hasta que

Este ciclo se utiliza cuando se sabe que el cuerpo del ciclo se va a ejecutar al menos una vez. La condición se evalúa al finalizar de ejecutarse las instrucciones que conforman el cuerpo del ciclo, si es verdadera, se repite la ejecución desde donde lo indica la palabra clave **hacer**, en caso contrario, se rompe el ciclo. Obsérvese que la única diferencia entre el ciclo **Mientras-Hacer** visto en la sección IV.2 es el momento en que se realiza la evaluación de la condición. El formato del ciclo **Repetir-Hasta** es mostrado en la Fig. 4.8.

Repetir

// Bloque de instrucciones a realizar en cada iteración

Hasta Que condición

Figura 4.8: Estructura del ciclo **Repetir-Hasta**

Ejemplo 4.24. Adivinar el número mágico (juego).

Se considera un valor oculto al usuario (llamado el número mágico) en un rango determinado. El juego consiste en que el usuario tratará de adivinarlo. Se requiere de un ciclo que se repeta mientras el usuario no haya adivinado el valor oculto. En el cuerpo del ciclo se le pide al usuario que proporcione el valor que cree que es el número mágico, `usuario`, y si no lo adivina se le proporcionan pistas sobre si el número mágico es mayor o menor al valor proporcionado y se actualiza el rango de valores donde el usuario puede buscar, utilizando las variables `linf` y `lsup`. En el Algoritmo 32 se muestra una posible implementación en pseudocódigo. En dicho algoritmo el rango de valores para el número mágico es del 1 al 20.

Algoritmo 4.26: Adivinar el número mágico (juego).

```

1 // Nombre: NumeroMagico.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso NumeroMagico
6   Definir NumMagico, usuario, LimInf, LimSup Como Enteros
7   Escribir "Este programa implementa el juego del número magico"
```

```

8   LimInf := 1
9   LimSup := 20
10  NumMagico := Aleatorio(1,20)
11  Repetir
12      Escribir Sin Saltar "Cual crees que es el número mágico? [",LimInf,"-",LimSup,"]="
13      Leer usuario
14      Si usuario<NumMagico Entonces
15          Escribir "El número mágico es mayor que ",usuario
16          LimInf := usuario
17      SiNo
18          Escribir "El número mágico es menor que ",usuario
19          LimSup := usuario
20      FinSi
21  Hasta Que usuario=NumMagico
22      Escribir "ADIVINASTE!!"
23  FinProceso

```

Un uso común que se le da a este ciclo es para controlar que el usuario proporcione la información adecuada. Así, por ejemplo, si queremos asegurarnos de que se proporcione una respuesta adecuada a una pregunta dada, utilizaremos este ciclo como "filtro" para que no se deje continuar el algoritmo hasta que se proporcione la respuesta correcta. Una situación como la mencionada se muestra en el Algoritmo 4.27. En éste, se solicita una respuesta que indique si se desea iniciar el juego, si la respuesta dada no es una 's' o 'S' o 'n' o 'N' se escribe un mensaje y se repite la solicitud y lectura de la respuesta, ya que la condición del ciclo **Repetir-Hasta** así lo indica. Una vez proporcionada la respuesta esperada se rompe el ciclo y se procede a revisar si la respuesta fue 's' o 'S' para iniciar el juego o, en caso contrario, despedirse del usuario.

Algoritmo 4.27: Inicio de juego.

```

1  // Nombre: InicioJuego.psc
2  // Autor: Irene Rodríguez y Edelmira Rodríguez
3  // Descripción:
4
5  Proceso InicioJuego
6      Definir resp Como Caracter
7
8      Repetir
9          Limpiar Pantalla
10         Escribir "Quieres iniciar el juego? (s/n) "
11         Leer resp
12
13         Si resp<>'n' y resp<>'N' y resp<>'s' y resp<>'S' Entonces
14             Escribir Sin Saltar "Se espera s/n "
15             Esperar Tecla
16         FinSi
17     Hasta Que resp='n' o resp='N' o resp='s' o resp='S'
18

```

```

19  Si resp='s' o resp='S' Entonces
20      Escribir "Comencemos a jugar"
21  SiNo
22      Escribir "Adios"
23  FinSi
24  FinProceso

```

En ocasiones pueden presentarse problemas en donde deba darse al usuario una lista de opciones en un menú, como en los Algoritmos 3.18 y 3.17, permitiendo que elija una de ellas para que se realice algún proceso después. A menudo, el menú va acompañada de una opción de salida del algoritmo, de tal manera que todo el cuerpo del algoritmo pueda ejecutarse repetidamente hasta que el usuario decida salir del mismo. A continuación, presentamos un ejemplo de este tipo de problema.

Ejemplo 4.25. Piedra, papel o tijeras (juego)..

Jugar hasta que el usuario decida salir y al terminar que se indique el número de juegos ganados por la máquina y por el jugador, así como el número de empates. En este ejemplo utilizaremos el Algoritmo 3.18 modificándolo para que se repita todo el cuerpo del algoritmo hasta que el jugador decida terminar. Para esto, agregaremos una opción al menú, salir, que será la opción que indicará que deberá romperse el ciclo y terminar. Para indicar el número de juegos ganados por el jugador, la máquina y los empates utilizamos tres contadores que se incrementan en uno según sea el caso. Ver el Algoritmo 4.28.

Algoritmo 4.28: Inicio de juego.

```

1  // Nombre: PPT_conCiclo.psc
2  // Autor:  Irene Rodríguez y Edelmira Rodríguez
3  // Descripción:
4
5  Proceso PPT_conCiclo
6      Definir jugador, maquina Como Entero
7
8      Repetir
9          Repetir
10             Limpiar Pantalla
11             Escribir "Este programa implementa el juego de Piedra Papel o Tijeras (PPT)"
12             Escribir "Teclea el número que corresponde a tu selección"
13             Escribir "1) Piedra"
14             Escribir "2) Papel"
15             Escribir "3) Tijeras"
16             Escribir "4) Salir"

```

```

17      Leer jugador
18      Hasta Que jugador>=1 y jugador<=4
19
20      Si jugador<>4 Entonces
21          // selección de la máquina, regresa un valor entero 1, 2 o 3
22          maquina := azar(3)+1
23
24      Si jugador=1 Entonces
25          Escribir "El jugador eligió PIEDRA"
26      SiNo
27          Si jugador=2 Entonces
28              Escribir "El jugador eligió PAPEL"
29          SiNo
30              Escribir "El jugador eligió TIJERAS"
31          FinSi
32      FinSi
33
34      Si maquina=1 Entonces
35          Escribir "La máquina eligió PIEDRA"
36      SiNo
37          Si maquina=2 Entonces
38              Escribir "La máquina eligió PAPEL"
39          SiNo
40              Escribir "La máquina eligió TIJERAS"
41          FinSi
42      FinSi
43
44
45      Si maquina=jugador Entonces
46          Escribir "-- Empate --"
47      SiNo
48          Segun jugador
49              1: // el jugador selecciona piedra
50                  Segun maquina
51                      2: // la máquina selecciona papel
52                          escribir "      ----> Gana la maquina "
53                      3: // la máquina selecciona tijeras
54                          escribir "      ----> Gana el jugador"
55                  FinSegun
56              2: // papel
57                  Segun maquina
58                      1: //piedra
59                          escribir "      ----> Gana el jugador "
60                      3: // tijeras
61                          escribir "      ----> Gana la maquina "
62                  FinSegun
63              3: // tijeras
64                  Segun maquina
65                      1: // piedra
66                          escribir "      ----> Gana la máquina "
67                      2: // papel
68                          escribir "      ----> Gana el jugador"
69                  FinSegun
70          FinSegun
71      FinSi
72      FinSi

```

```

73     Esperar Tecla
74     Hasta Que jugador=4
75 FinProceso

```

Obsérvese el segmento siguiente:

```

Repetir
    Limpiar Pantalla
    Escribir "Este programa implementa el juego de Piedra Papel o Tijeras (PPT)"
    Escribir "Teclea el numero que corresponde a tu seleccion"
    Escribir "1) Piedra"
    Escribir "2) Papel"
    Escribir "3) Tijeras"
    Escribir "4) Salir"
    Leer jugador
Hasta Que jugador>=1 y jugador<=4

```

El uso del ciclo Repetir-Hasta obliga a que el usuario proporcione un valor en el rango de opciones presentadas ya que si jugador es menor que 1 o mayor que 4 se repetirá la solicitud y lectura de una selección para el usuario.

En algunas situaciones el ciclo Repetir-Hasta se adapta mejor a las necesidades del algoritmo que resuelve un problema. Ilustraremos esto con el Ejemplo 4.14 donde se plantea el problema de calcular el promedio de calificaciones utilizando un valor centinela.

Recuérdese que se utilizó el ciclo Mientras-Hacer para leer las calificaciones, siendo necesario leer la primera de ellas fuera del ciclo para poder usar esa primera calificación para controlar el ciclo. Por lo tanto, es posible que el ciclo no se ejecute (si calif=-1 en la primera lectura), sin embargo en el código deben incluirse las instrucciones para solicitar la calificación dos veces (fuera del ciclo y dentro de él). A continuación se ilustra esta situación, a manera de recordatorio.

```

Escribir Sin Saltar "Escribe una calificacion [para terminar, teclea un -1]"
Leer calif

Mientras calif != -1 Hacer
    // codigo
    Escribir "Escribe la siguiente calificacion [para terminar, teclea un -1]"
    Leer calif
FinMientras

```

En los casos donde es necesario ejecutar el cuerpo del ciclo al menos una vez, se recomienda utilizar el ciclo Repetir-Hasta. A continuación, se muestra un algoritmo para resolver el problema planteado, utilizando el ciclo presentado en esta sección.

Algoritmo 4.29: Calcular el promedio de calificaciones.

```
1 // Nombre: UsandoCentinela4.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4
5 Proceso UsandoCentinela4
6   Definir calif, suma Como Real
7   Definir contador Como Entero
8
9   contador := 0
10  suma := 0
11
12  Escribir "Este programa calcula el promedio de una serie de calificaciones dadas "
13
14  Repetir
15    Escribir Sin Saltar "Escribe una calificación [para terminar, teclea un -1]"
16    Leer calif
17
18    Si calif >= 0 y calif <= 100 Entonces
19      contador := contador + 1
20      suma := suma + calif
21    FinSi
22  Hasta Que calif=-1
23
24  Si contador>0 Entonces
25    Escribir "El promedio es ", suma/contador
26  FinSi
27 FinProceso
```

Ejemplo 4.26. Una trivia es un juego de preguntas y respuestas. En cada pregunta se dan varias opciones de respuesta, si el usuario elige la respuesta correcta, ganará un punto.

En el Algoritmo 4.30 se muestra una implementación del juego mencionado, con tres preguntas y cuatro respuestas posibles en cada una. AL finalizar el juego, se muestran los ppuntos ganados.

Algoritmo 4.30: Juego de preguntas y respuestas

```
1 // Nombre: Trivia.psc
2 // Autor: Irene Rodríguez y Edelmira Rodríguez
3 // Descripción:
4 //
```

```

5 Proceso TRIVIA
6   Definir respuesta Como Caracter
7   Definir respuestaCorrecta Como Caracter
8   Definir puntos Como Entero
9   puntos := 0
10
11  Escribir "Bienvenido al juego de Trivia!"
12
13  // Primera pregunta
14  Escribir "Pregunta 1:Cuál es la capital de Francia?"
15  Escribir "A) Berlín"
16  Escribir "B) Madrid"
17  Escribir "C) Paris"
18  Escribir "D) Roma"
19  Leer respuesta
20  respuesta:=Mayusculas(respuesta)
21  respuestaCorrecta := "C"
22  Si respuesta = respuestaCorrecta Entonces
23    Escribir "Correcto!"
24    puntos := puntos + 1
25  Sino
26    Escribir "Incorrecto. La respuesta correcta es C) Paris."
27  FinSi
28
29  // Segunda pregunta
30  Escribir "Pregunta 2: Cual es el océano más grande?"
31  Escribir "A) Atlántico"
32  Escribir "B) Índico"
33  Escribir "C) Ártico"
34  Escribir "D) Pacífico"
35  Leer respuesta
36  respuesta:=Mayusculas(respuesta)
37  respuestaCorrecta := "D"
38  Si respuesta = respuestaCorrecta Entonces
39    Escribir "Correcto!"
40    puntos := puntos + 1
41  Sino
42    Escribir "Incorrecto. La respuesta correcta es D) Pacífico."
43  FinSi
44
45  // Tercera pregunta
46  Escribir "Pregunta 3: Quién escribió Cien años de soledad?"
47  Escribir "A) Julio Cortazar"
48  Escribir "B) Gabriel García Márquez"
49  Escribir "C) Mario Vargas Llosa"
50  Escribir "D) Jorge Luis Borges"
51  Leer respuesta
52  respuesta:=Mayusculas(respuesta)
53  respuestaCorrecta := "B"
54  Si respuesta = respuestaCorrecta Entonces
55    Escribir "Correcto!"
56    puntos := puntos + 1
57  Sino
58    Escribir "Incorrecto. La respuesta correcta es B) Gabriel García Marquez."
59  FinSi
60

```

```
61 |   Escribir "Has terminado el juego. Tu puntaje es: ", puntos, " de 3."  
62 | FinProceso
```

Posibles mejoras al juego

- Presentar categorías de preguntas, el usuario elige la categoría donde quiere jugar. Por ejemplo, pueden ser de Literatura, Cine, Geografía,...
- Incluir niveles de dificultad de las preguntas. A mayor nivel, mayor será la dificultad de las preguntas.

A manera de resumen mencionaremos los usos más frecuentes de cada ciclo:

- El ciclo **Mientras-Hacer** se utiliza cuando alguna operación a realizar repetidamente necesita supervisión previa ya que la revisión de la condición que lo controla se ejecuta antes que el cuerpo del ciclo. Es ideal utilizarlo si no se conoce el número de repeticiones necesarias para el proceso.
- El ciclo **Para-Hacer** se utiliza cuando tenga que repetirse la ejecución de un proceso un determinado número de veces. Este ciclo está controlado con una operación de inicialización, condición e incremento, aplicadas a un contador.
- Se utiliza el ciclo **Repetir-Hasta** cuando la ejecución de un proceso deberá realizarse por lo menos una vez ya que la revisión de la condición que lo controla se ejecuta después del cuerpo del ciclo.

Referencias

- Deitel, HM & Deitel, PJ. (2003). *C++ Cómo Programar* (4a ed.). Cd. de México: Pearson-Prentice Hall.
- Joyanes Aguilar, L. (2000). *Programación en C++. Algoritmos, Estructuras de Datos y Objetos*. Madrid: Mc Graw Hill.
- Kernighan, BW & Ritchie, DM. (1978). *The C Programming Language*. London: Prentice Hall.
- McConnell, S. (1993). *Code Complete: A Practical Handbook of Software Construction*. Redmond: Microsoft Press.
- Savitch, W. (2007). *Resolución de Problemas con C++* (5a ed.). Cd. de México: Pearson Educación.
- Schildt, H. (2001). *C Manual de Referencia* (4a ed.). Madrid: Mc Graw Hill.
- Stroustrup, B. (2002). *El Lenguaje de Programación C++* (Ed. especial). Madrid: Addison Wesley.

Lista de algoritmos

2.1. Solicitud y captura de datos	14
2.2. Solicitud y captura de datos	15
2.3. Solicitud y captura de datos	16
2.4. Área de un triángulo.	21
2.5. Área de un triángulo.	22
2.6. Área de un círculo.	24
2.7. Área de un círculo.	24
2.8. Operaciones enteras.	25
2.9. Operaciones enteras.	25
3.1. Menor de dos números.	35
3.2. Menor de dos números usando una variable auxiliar.	36
3.3. Determinar si un número entero dado es par.	37
3.4. Contratación de un empleado.	39
3.5. Encontrar el mayor de dos números dados.	41
3.6. Encontrar el mayor de dos números dados.	42
3.7. Encontrar el mayor de tres números dados.	43

3.8. Encontrar el mayor de tres números dados.	44
3.9. Encontrar el mayor de tres números dados.	45
3.10. Contratación de un empleado (usando AND).	46
3.11. Contratación de un empleado (usando OR).	47
3.12. Contratación de un empleado.	47
3.13. Saludando al usuario por su nombre.	49
3.14. Saludando al usuario por su nombre y edad.	50
3.15. Saludando al usuario por su nombre, edad y peso.	52
3.16. Índice de masa corporal.	53
3.17. Día de la semana (v2).	55
3.18. Día de la semana (v1).	57
3.19. Día de la semana (v2).	58
3.20. Día de la semana (v3).	60
3.21. Menu de opciones.	61
3.22. Lanzando dados.	63
3.23. Juego El 7.	66
3.24. Juego El 7.	67
3.25. Juego El 7.	67
3.26. Piedra, papel y tijeras.	71
3.27. Concatenar cadenas.	72
4.1. Promedio de grupo.	81
4.2. Encontrar el mayor de 8 numeros dados.	82
4.3. Suma de los pares en el intervalo mencionado.	83

4.4. Suma de los pares en el intervalo dado.	83
4.5. Suma de los pares y los impares en el intervalo dado.	84
4.6. Suma de los pares y los impares en el intervalo dado.	85
4.7. Contar el numero de pares leídos.	86
4.8. Cálculo del valor de PI a partir de la serie infinita dada.	87
4.9. Cálculo del valor de PI a partir de la serie infinita dada.	88
4.10. Mutiplicar los impares leídos (1).	89
4.11. Mutiplicar los impares leídos (2).	89
4.12. Factorial de n.	90
4.13. Identificar si un número dado es primo.	93
4.14. Uso de un valor centinela (promedio de calificaciones).	95
4.15. Uso de un carácter como valor centinela.	97
4.16. Factorial de n.	99
4.17. Lanzamiento de una moneda 100 veces.	100
4.18. Generar la sucesión de Fibonacci	101
4.19. Determinar si un número entero positivo dado es primo.	102
4.20. Calcular una integral numérica inferior.	104
4.21. Calcular una integral numérica inferior.	105
4.22. Dibuja un rectangulo.	106
4.23. Dibuja un Triángulo.	107
4.24. Dibuja un Triangulo (2).	107
4.25. Dibuja un Triangulo (3).	109
4.26. Adivinar el número mágico (juego).	110

4.27. Inicio de juego.	111
4.28. Inicio de juego.	112
4.29. Calcular el promedio de calificaciones.	115
4.30. Juego de preguntas y respuestas	115

Lista de cuadros

2.1. Palabras reservadas del PseInt	5
2.2. Operadores aritméticos	17
2.3. Precedencia y asociatividad de los operadores aritméticos . .	20
2.4. Operadores de relación o relacionales.	27
2.5. Tabla de verdad del operador lógico AND.	28
2.6. Tabla de verdad del operador lógico OR.	29
2.7. Tabla de verdad del operador lógico NO.	29
2.8. Leyes de DeMorgan para la lógica.	30
2.9. Precedencia de los operadores.	30
3.1. Reglas del juego PPT.	70

Lista de figuras

2.1.	Diagrama para la lectura de datos.	9
2.2.	Diagrama para la lectura de datos.	11
2.3.	Diagrama de flujo para el Algoritmo 2.1.	14
2.4.	Símbolo de la asignación en los diagramas de flujo.	18
2.5.	Diagrama de flujo para el Algoritmo 2.4.	22
3.1.	Formato de la selección simple.	33
3.2.	Diagrama de flujo.	36
3.3.	Diagrama de flujo del Algoritmo 3.3	38
3.4.	Formato de la selección doble.	40
3.5.	Formato de la selección doble.	40
3.6.	Diagrama de flujo del Algoritmo 3.5.	41
3.7.	Formato de la selección múltiple.	59
4.1.	Estructura del ciclo Mientras-Hacer	73
4.2.	Estructura del ciclo Mientras-Hacer	98
4.3.	Aproximación al área bajo la curva de $f(x)$	103

4.4.	Rectángulo	106
4.5.	Triangulo1	107
4.6.	Triangulo 2	108
4.7.	Triangulo 3	109
4.8.	Estructura del ciclo Repetir-Hasta	110